

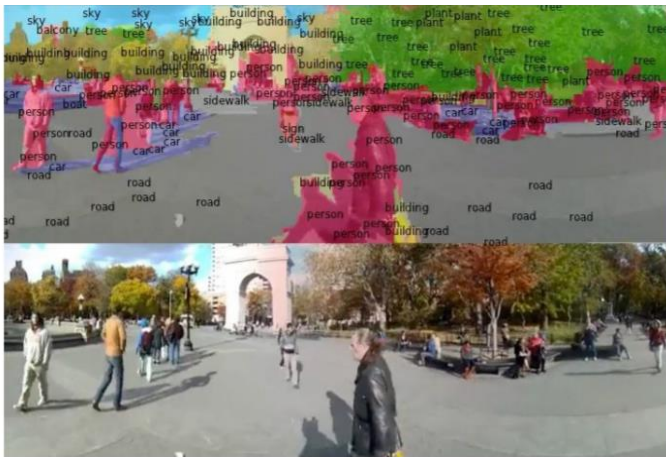
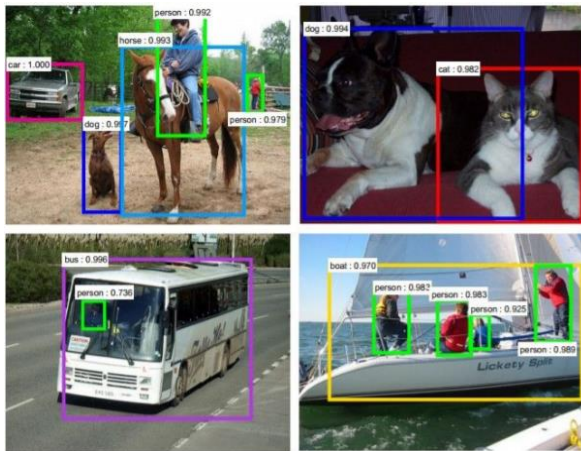


QpiAI™ - Joint AI Training and Certification Program

Deep Learning, Convolutional Neural Networks
and CNN Architectures

Sachin Kumar

Motivation



Data

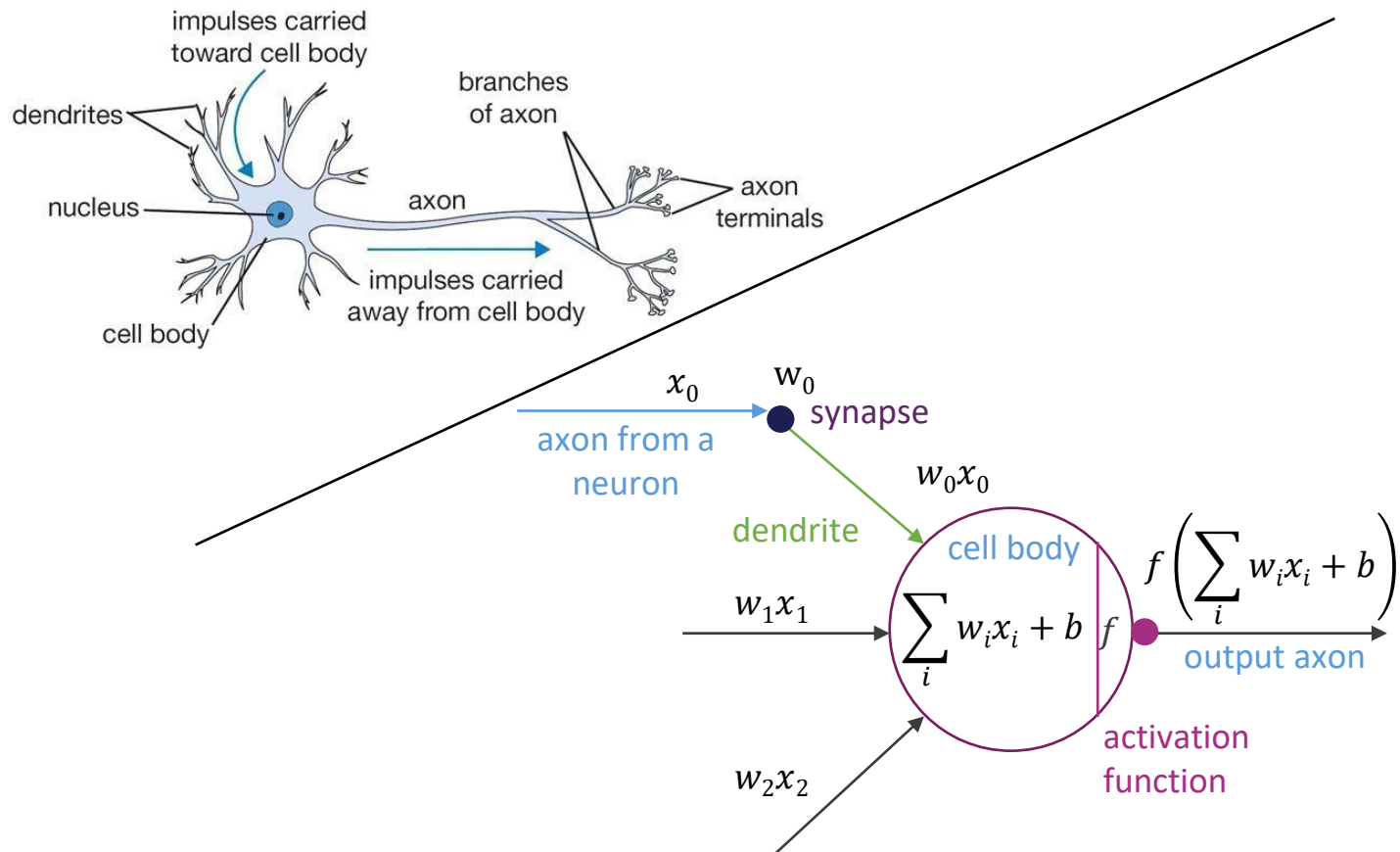
+

Algorithms

+

Compute Power

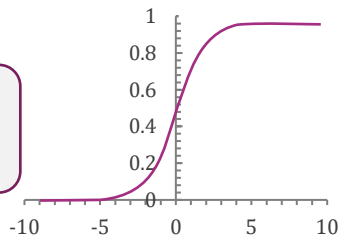
Neural Networks: A single neuron



Activation Functions

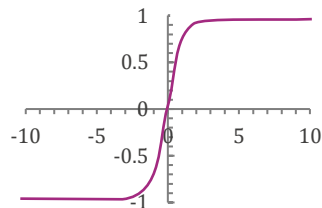
Sigmoid

$$\sigma(x) = 1/(1 + e^{-x})$$



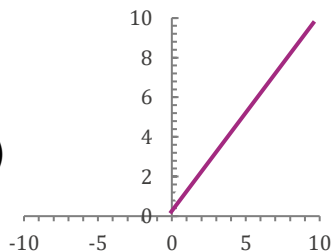
tanh

$$\tanh(x)$$



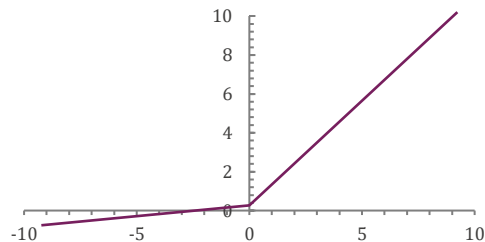
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

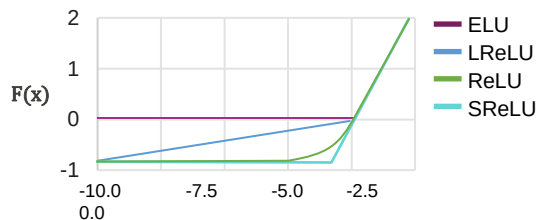


Maxout

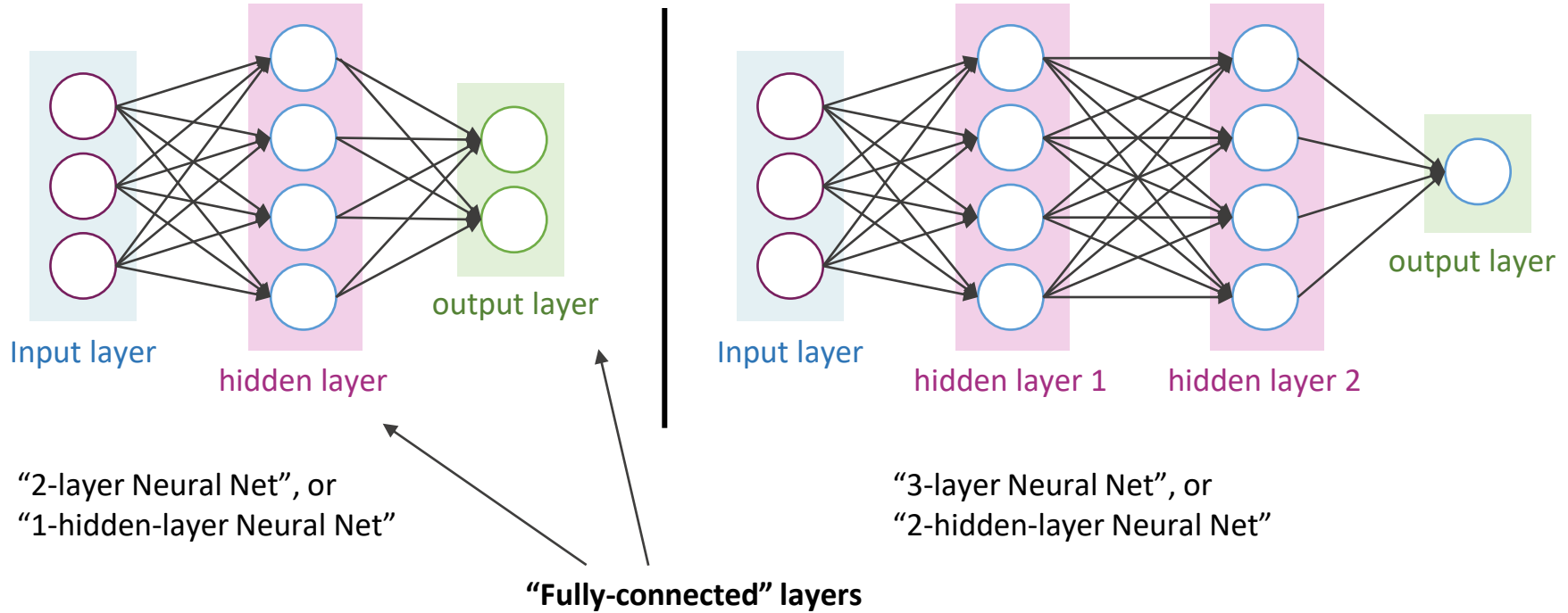
$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

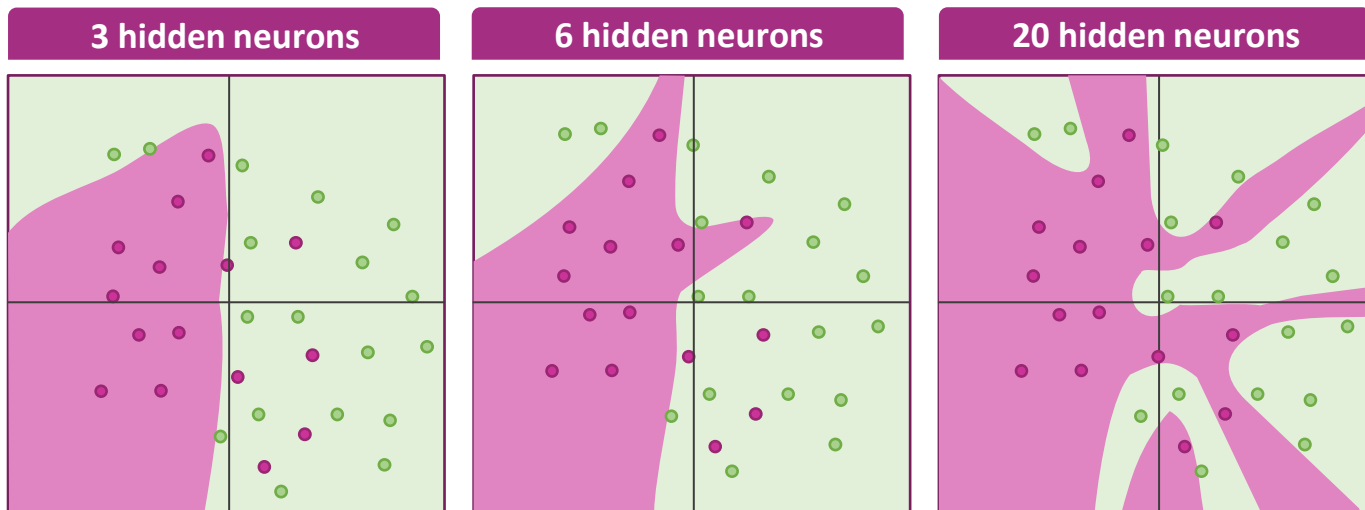
$$f(x) = \begin{cases} x & \text{if } x > 0 \\ \sigma(\exp(x)) - 1 & \text{if } x \leq 0 \end{cases}$$



Neural Networks: Basic Architecture



Setting the number of layers and their sizes



More Neurons give greater capacity to the network

Parametric approach : Linear Classifier



[32 x 32 x 3]
array of numbers
0...1 (3072 numbers total)

Image Parameters

$$f(\mathbf{x}, \mathbf{W})$$



10 numbers,
indicating class
scores

Parametric approach : Linear Classifier



$[32 \times 32 \times 3]$
array of numbers

$$f(\mathbf{x}, \mathbf{W}) = \mathbf{W}\mathbf{x} + \mathbf{b}$$

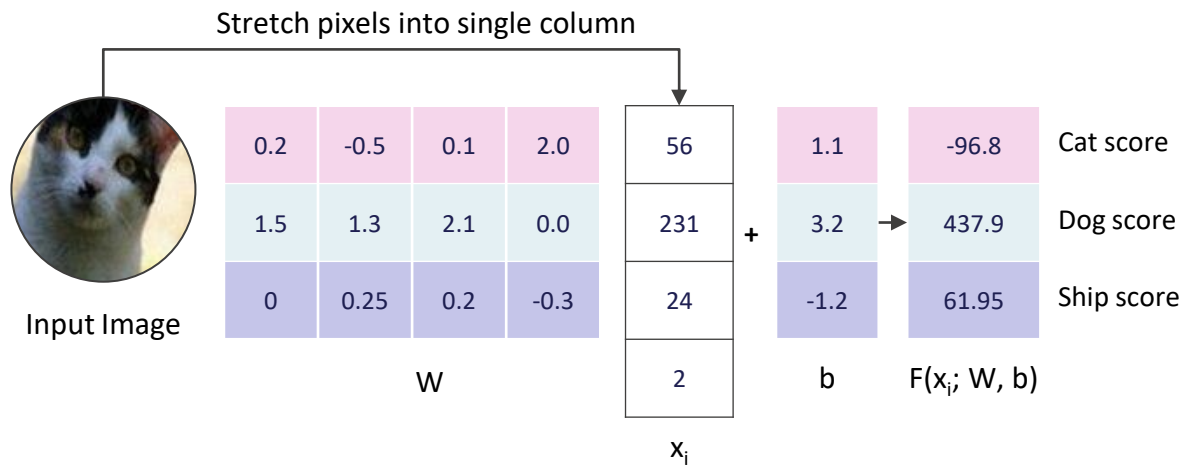
$10 \times 1 \quad 10 \times 3072$

3072×1
10 numbers,
 indicating class
 scores

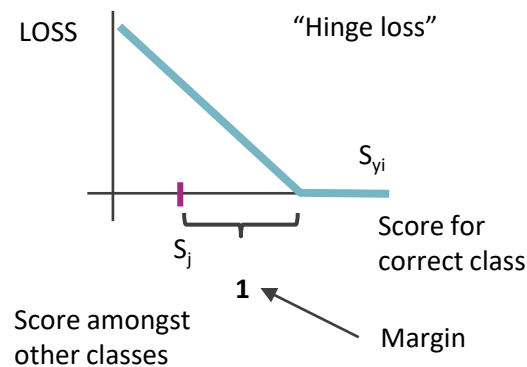
parameters, or “weights”

$$\text{Total Parameters} = 3072 \times 10 \quad (+\mathbf{b}) \quad 10 \times 1$$

Example with an image with 4 pixels, and 3 classes (Cat/Dog/Ship)



Interpreting Multiclass SVM loss



$$L_i = \sum_{j \neq y_i} \begin{cases} 0, & \text{if } s_{y_i} \geq s_j + 1 \\ s_j - s_{y_i} + 1, & \text{Otherwise} \end{cases}$$

$$= \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

Example of Training with a Multiclass SVM Loss

Suppose: 3 training examples, 3 classes.
With some W the scores $f(x_i, W) = Wx$ are:



Cat	3.2	1.3	2.2
Car	5.1	4.9	2.5
Frog	-1.7	2.0	-3.1
Losses	2.9		

Multiclass SVM loss

Given an example (x_i, y_i)

Where x_i is the image and

Where y_i is the (integer) label,

and using the shorthand for the scores vector: $s = f(x_i, W)$

The SVM loss has the form:

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

$$\begin{aligned} &= \max(0, 5.1 - 3.2 + 1) \\ &\quad + \max(0, -1.7 - 3.2 + 1) \\ &= \max(0, 2.9) + \max(0, -3.9) \\ &= 2.9 + 0 \\ &= 2.9 \end{aligned}$$

Example of Training with a Multiclass SVM Loss

Suppose: 3 training examples, 3 classes.
With some W the scores $f(x_i, W) = Wx$ are:



Cat	3.2	1.3	2.2
Car	5.1	4.9	2.5
Frog	-1.7	2.0	-3.1
Losses	2.9	0	

Multiclass SVM loss

Given an example (x_i, y_i)

Where x_i is the image and

Where y_i is the (integer) label,

and using the shorthand for the scores vector: $s = f(x_i, W)$

The SVM loss has the form:

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

$$\begin{aligned}
 &= \max(0, 1.3 - 4.9 + 1) \\
 &\quad + \max(0, 2.0 - 4.9 + 1) \\
 &= \max(0, -2.6) + \max(0, -1.9) \\
 &= 0 + 0 \\
 &= 0
 \end{aligned}$$

Example of Training with a Multiclass SVM Loss

Suppose: 3 training examples, 3 classes.
With some W the scores $f(x_i, W) = Wx$ are:



Cat	3.2	1.3	2.2
Car	5.1	4.9	2.5
Frog	-1.7	2.0	-3.1
Losses	2.9	0	10.9

Multiclass SVM loss

Given an example (x_i, y_i)

Where x_i is the image and

Where y_i is the (integer) label,

and using the shorthand for the scores vector: $s = f(x_i, W)$

The SVM loss has the form:

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

$$\begin{aligned}
 &= \max(0, 2.2 - (-3.1) + 1) \\
 &\quad + \max(0, 2.5 - (-3.1) + 1) \\
 &= \max(0, 5.3) + \max(0, 5.6) \\
 &= 5.3 + 5.6 \\
 &= 10.9
 \end{aligned}$$

Example of Training with a Multiclass SVM Loss

Suppose: 3 training examples, 3 classes.
With some W the scores $f(x_i, W) = Wx$ are:



Cat	3.2	1.3	2.2
Car	5.1	4.9	2.5
Frog	-1.7	2.0	-3.1
Losses	2.9	0	10.9

Multiclass SVM loss

Given an example (x_i, y_i)

Where x_i is the image and

Where y_i is the (integer) label,

and using the shorthand for the scores vector: $s = f(x_i, W)$

The SVM loss has the form:

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

and the full training loss is the mean over all examples in the training data:

$$L = \frac{1}{N} \sum_{i=1}^N L_i$$

$$L = (2.9 + 0 + 10.9)/3 \\ = 4.6$$

Softmax Classifier (Multinomial Logistic Regression)



Scores = unnormalized log probabilities of the classes.

$$P(Y = k|X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

where

$$S = f(x_i W)$$

Cat	3.2
Car	5.1
Frog	-1.7
Losses	2.9

Want to maximize the log likelihood, or (for a loss function) to minimize the negative log likelihood of the correct class:

$$L_i = -\log P(Y = y_i|X = x_i)$$

in summary:

$$L_i = -\log \left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}} \right)$$



$$L_i = -\log\left(\frac{e^s}{\sum_j e^{s_j}} y_i\right)$$

unnormalized probabilities

Cat	3.2
Car	5.1
Frog	-1.7

Unnormalized
log probabilities

exp

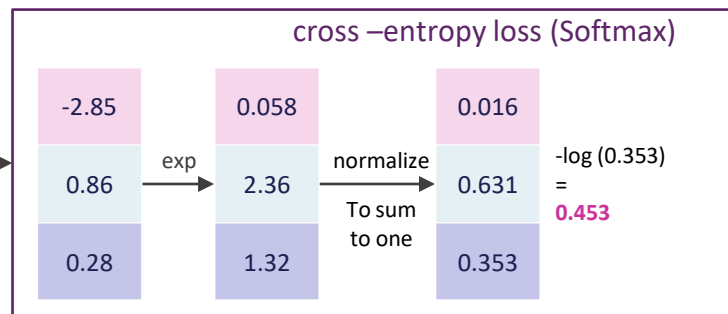
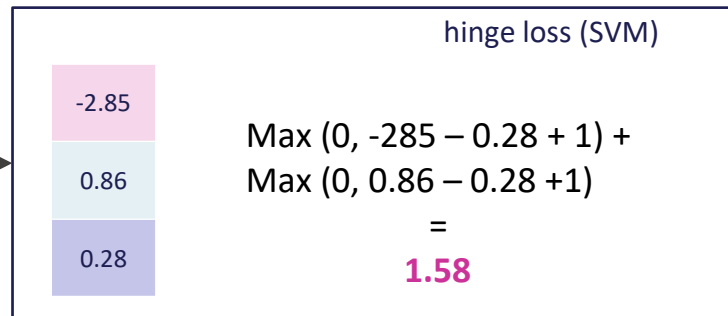
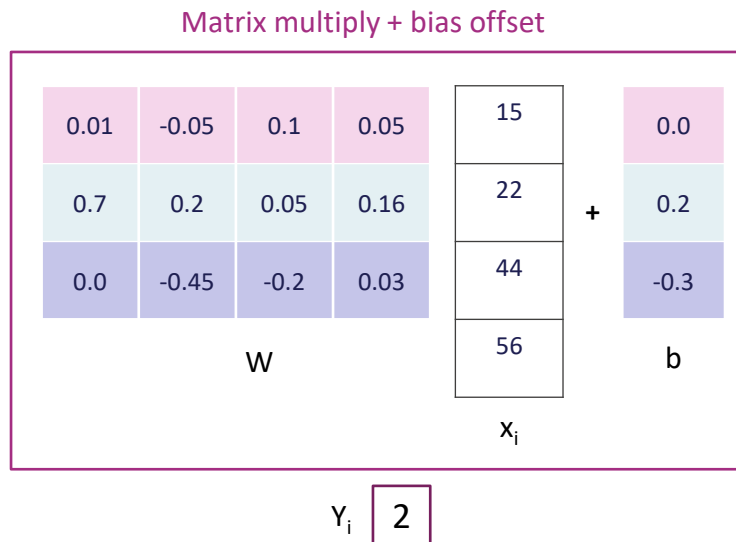
24.5
164.0
0.18

normalize

0.13
0.87
0.00

Probabilities

$$L_i = -\log(0.13) = 0.89$$



- We have some dataset of (X, y)
- We have a **score function S**
- We have a **loss function L**

$$L_i = -\log\left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}}\right)$$

= Softmax

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

= SVM

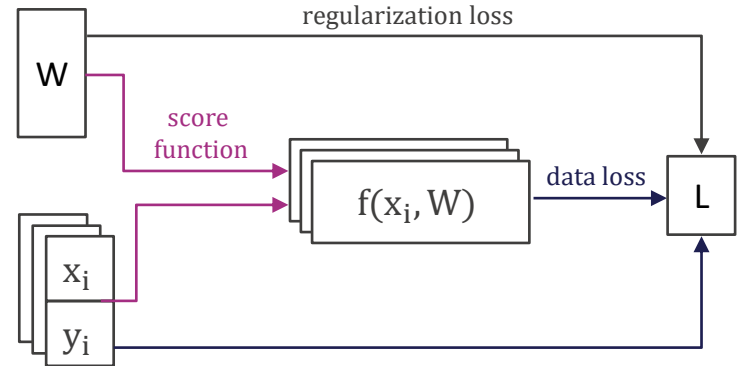
$$L = \frac{1}{N} \sum_{i=1}^N L_i + R(W)$$

= Full loss

For Optimization, we want:

$$\nabla_W L$$

$$S = f(x; W) = Wx$$



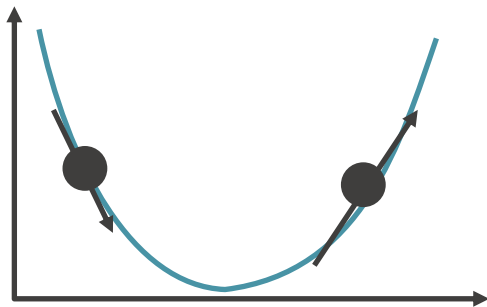
Gradient descent and Loss Function Landscape

An algorithm :

1. Find a direction v where $\mathcal{L}(\theta)$ decreases
2. $\theta \leftarrow \theta + av$

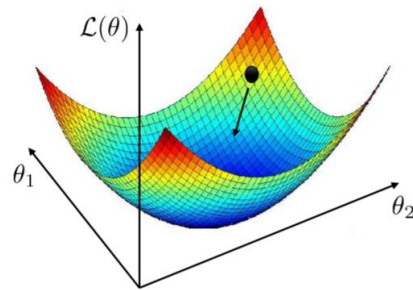
a is some small constant called “learning rate” or “step size”

Which way does $\mathcal{L}(\theta)$ decrease?



- Negative slope = go to the right
- Positive slope = go to the left
- In general :
 - For each dimension, go in the direction opposite the slope **along that dimension**

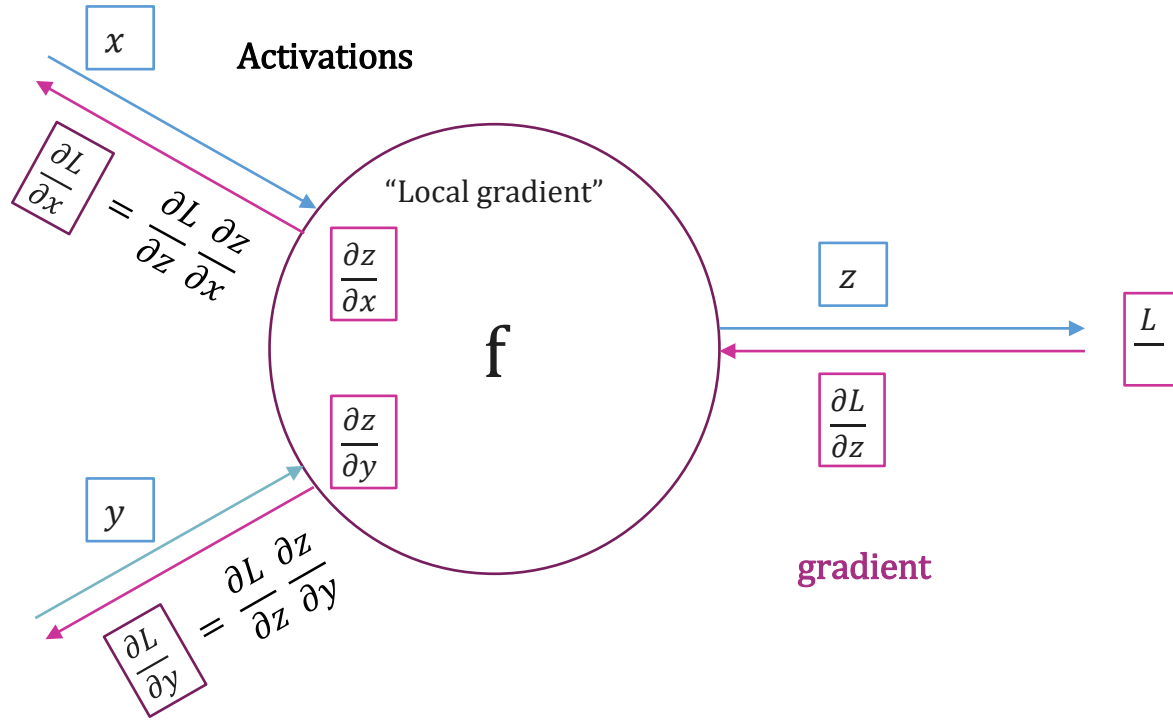
$$v_1 = - \frac{d\mathcal{L}(\theta)}{d\theta_1} \quad v_2 = - \frac{d\mathcal{L}(\theta)}{d\theta_2}$$



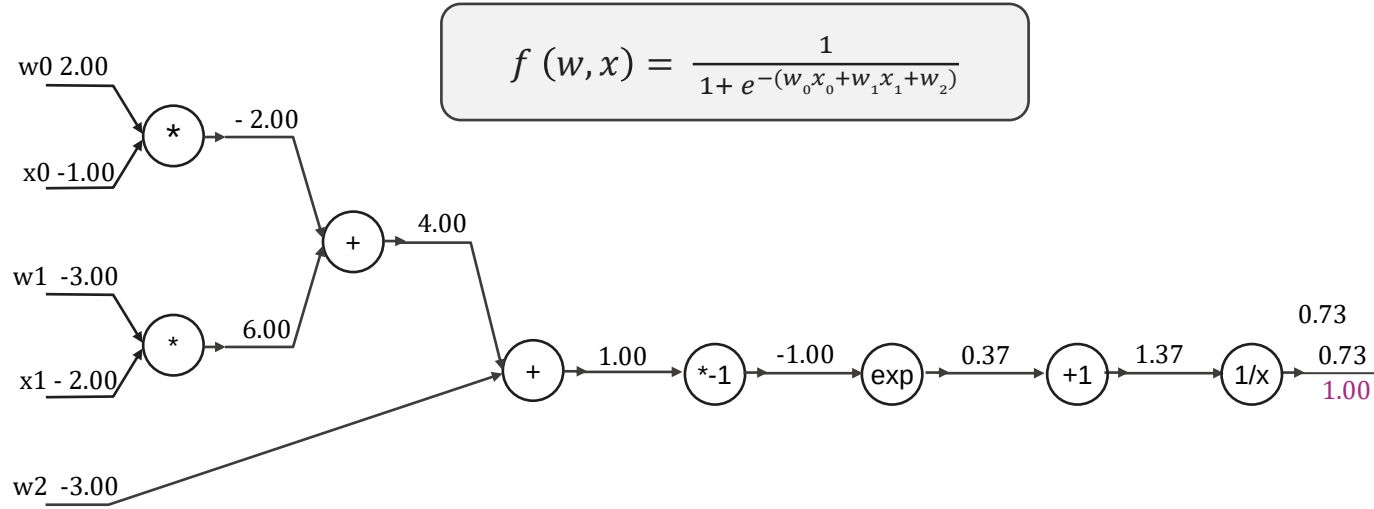
gradient :

$$\nabla_{\theta} \mathcal{L}(\theta) = \begin{bmatrix} \frac{d\mathcal{L}(\theta)}{d\theta_1} \\ \frac{d\mathcal{L}(\theta)}{d\theta_2} \\ \vdots \\ \frac{d\mathcal{L}(\theta)}{d\theta_n} \end{bmatrix}$$

Backpropagation



Simple example: Backpropagation through Sigmoid function

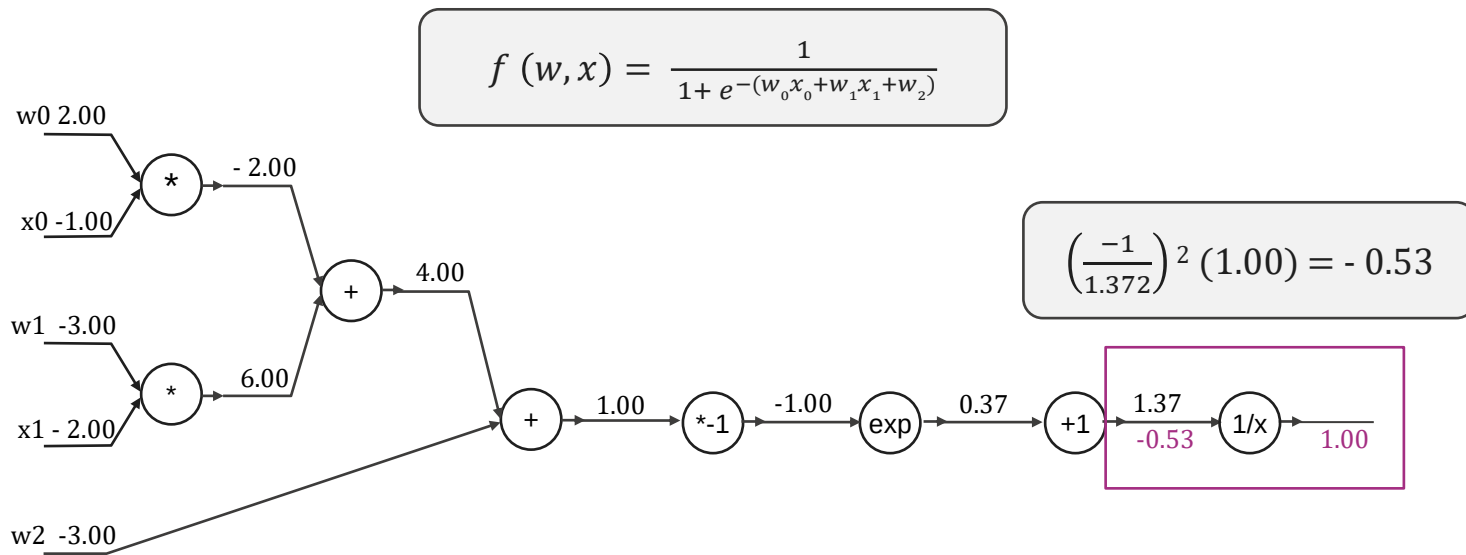


$$f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2 x_2)}}$$

$$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$$

Backpropagation



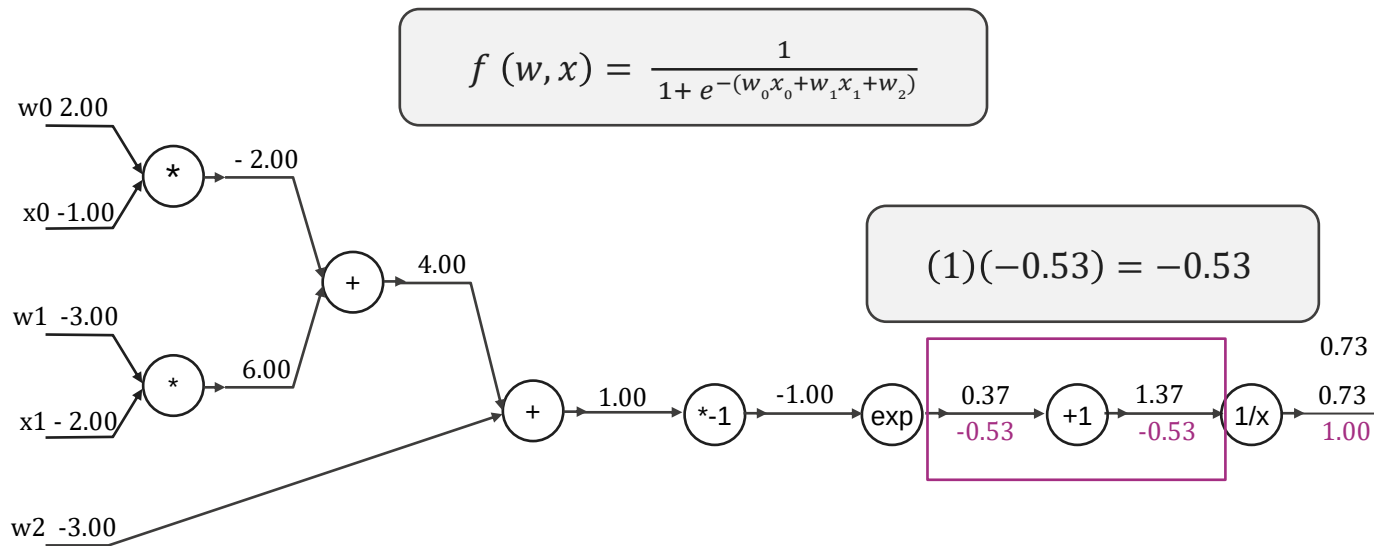
$$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$$

$$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -1/x^2$$

$$f_a(x) = ax \rightarrow \frac{df}{dx} = a$$

$$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$$

Backpropagation



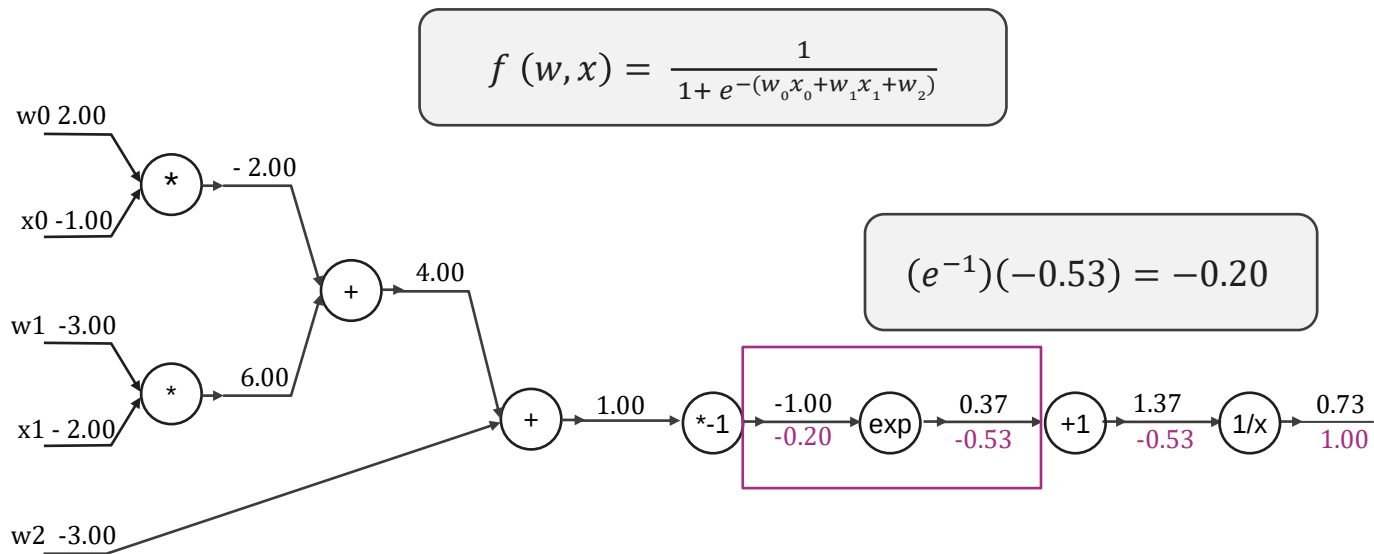
$$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$$

$$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -1/x^2$$

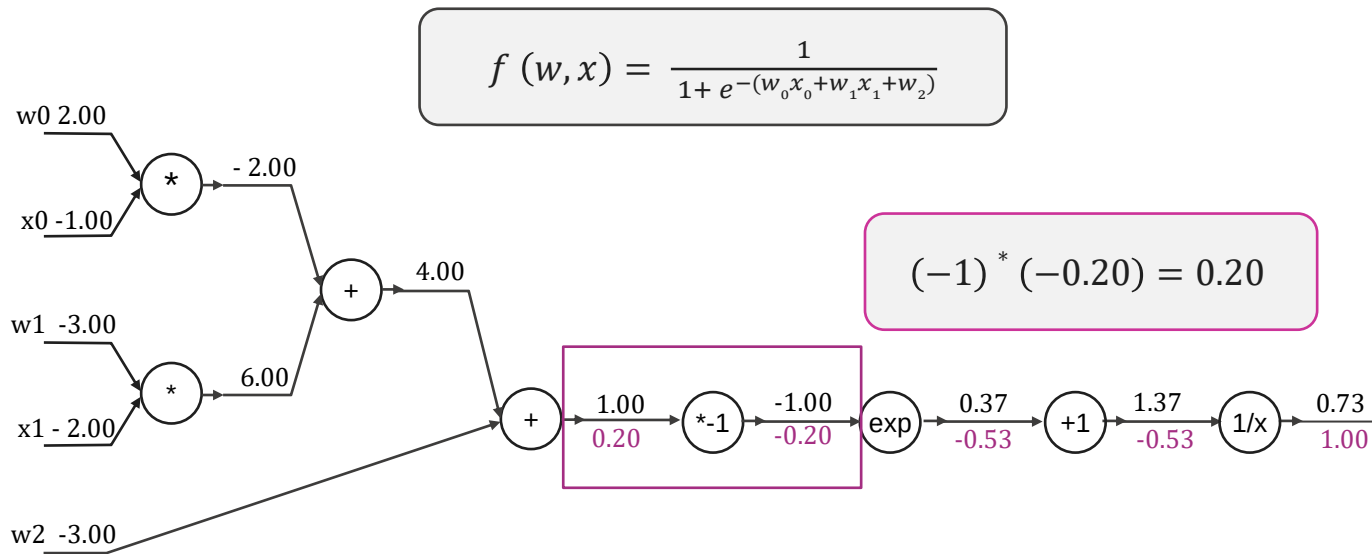
$$f_a(x) = ax \rightarrow \frac{df}{dx} = a$$

$$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$$

Backpropagation



Backpropagation



$$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$$

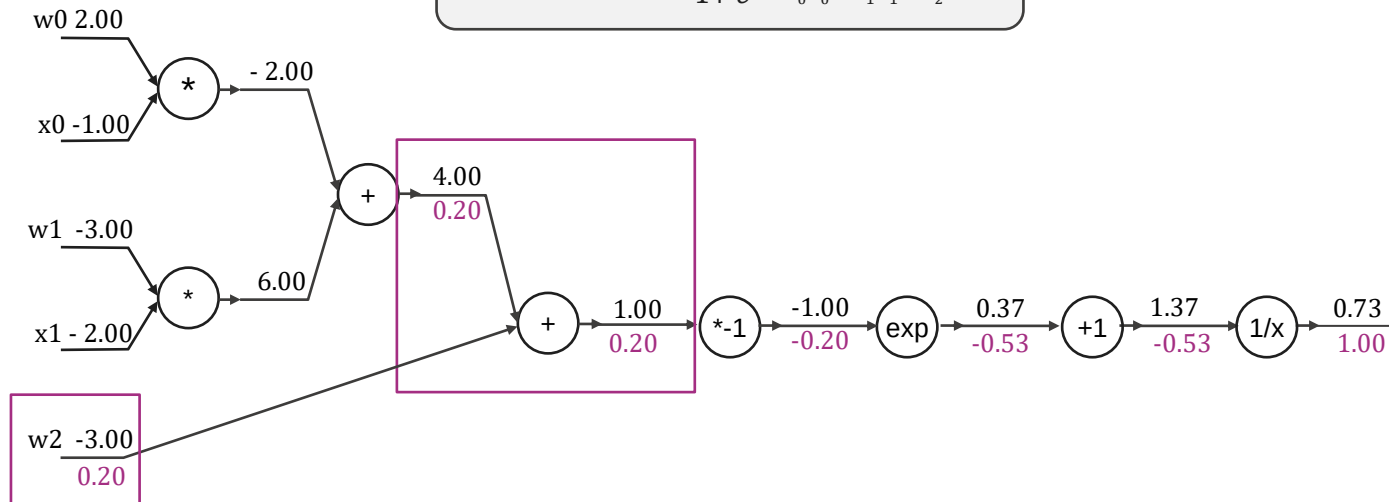
$$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -1/x^2$$

$$f_a(x) = ax \rightarrow \frac{df}{dx} = a$$

$$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$$

Backpropagation

$$f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$$



$$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$$

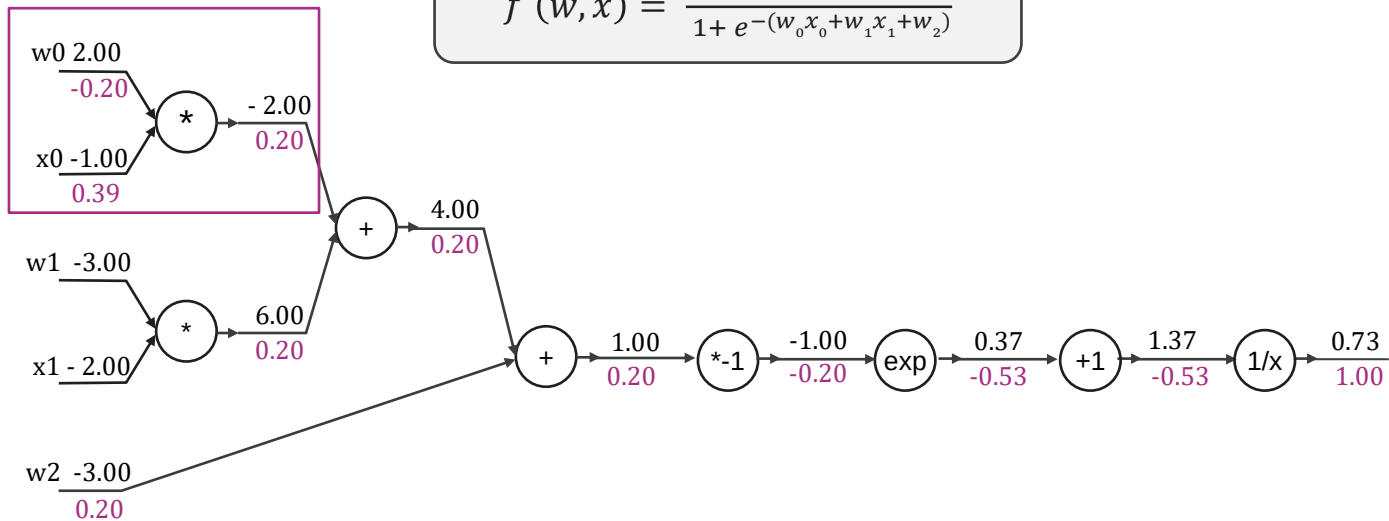
$$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -1/x^2$$

$$f_a(x) = ax \rightarrow \frac{df}{dx} = a$$

$$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$$

Backpropagation

$$f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2 x_2)}}$$



$$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$$

$$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -1/x^2$$

$$f_a(x) = ax \rightarrow \frac{df}{dx} = a$$

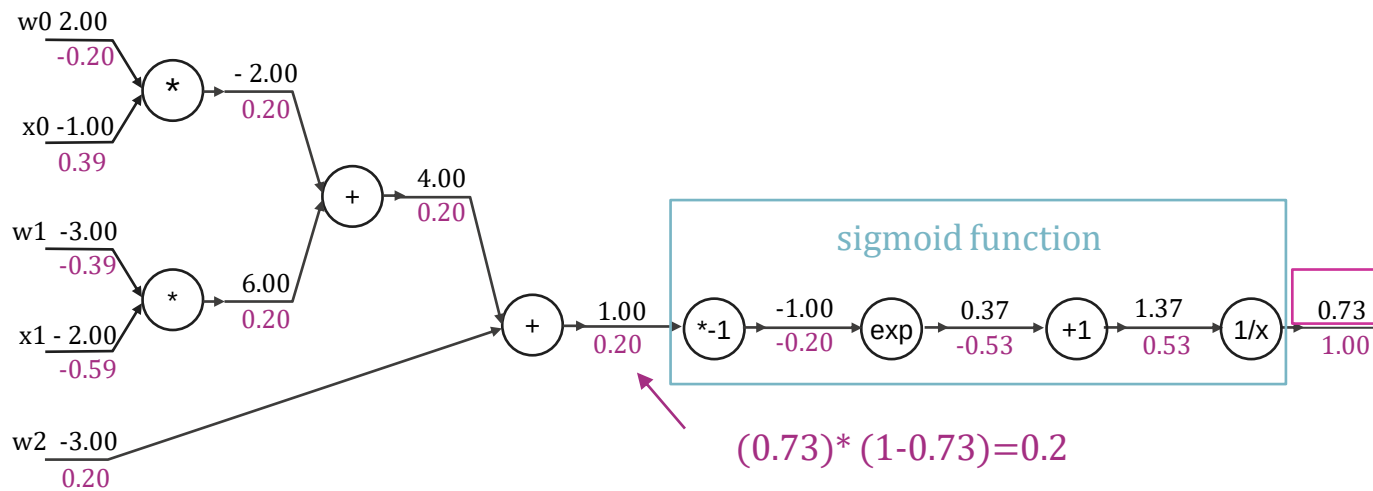
$$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$$

$$f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$$

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

sigmoid function

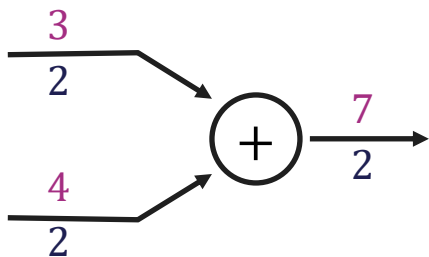
$$\frac{d\sigma(x)}{dx} = \frac{e^{-x}}{(1+e^{-x})^2} = \left(\frac{1+e^{-x}-1}{1+e^{-x}}\right) \left(\frac{1}{1+e^{-x}}\right) = (1 - \sigma(x))\sigma(x)$$



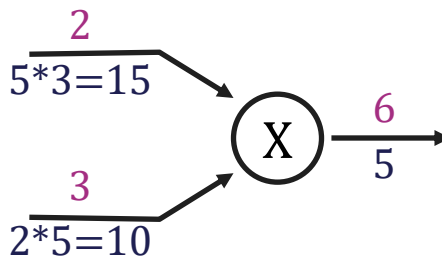
$$(0.73) * (1 - 0.73) = 0.2$$

Patterns in gradient flow

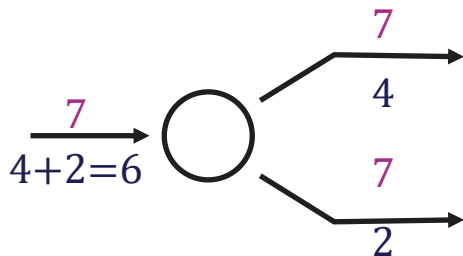
Add gate : gradient distributor



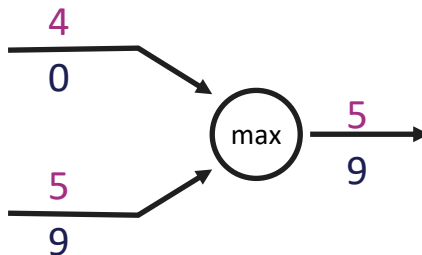
Multiply gate : swap multiplier



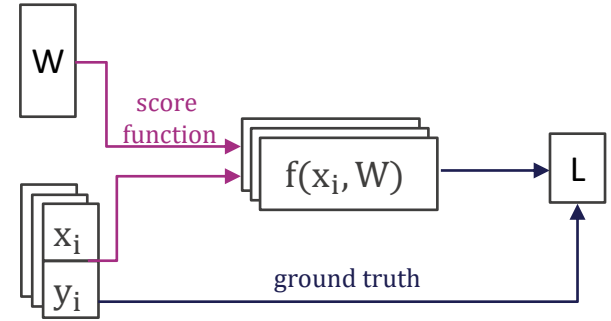
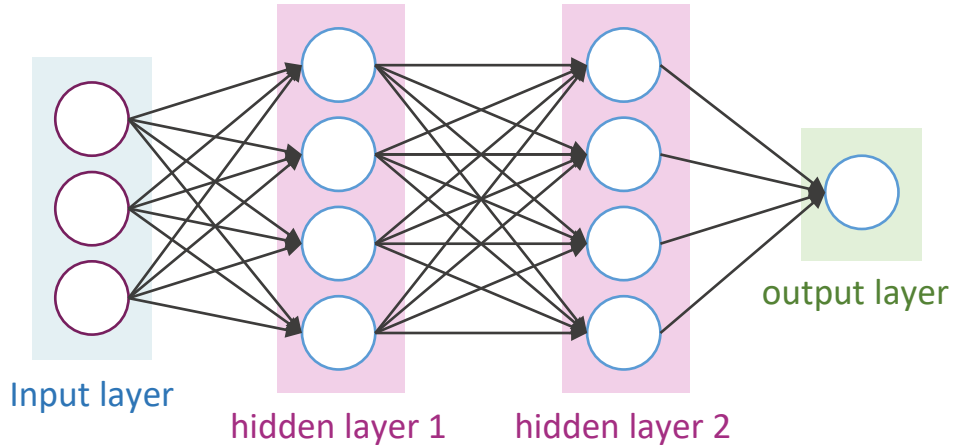
Copy gate : gradient adder



Max gate : gradient router



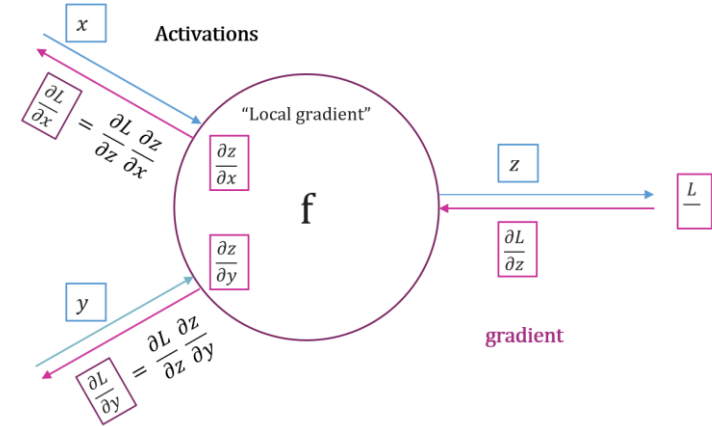
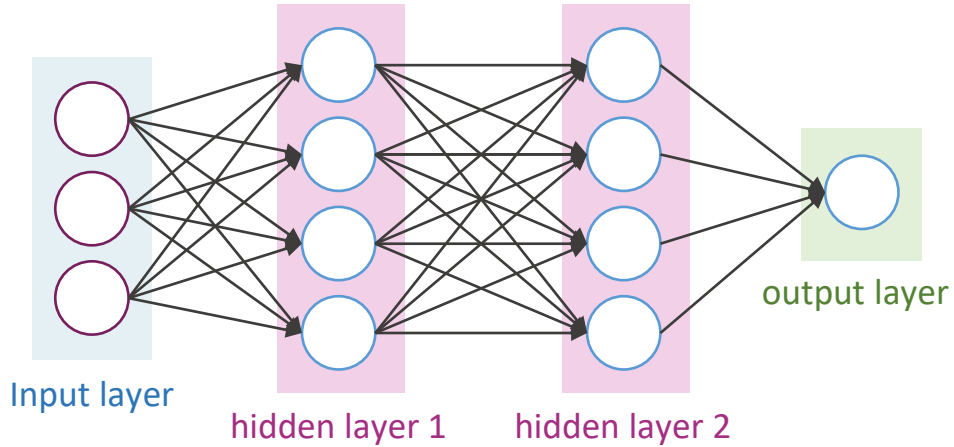
Summarising how Neural Networks Learn



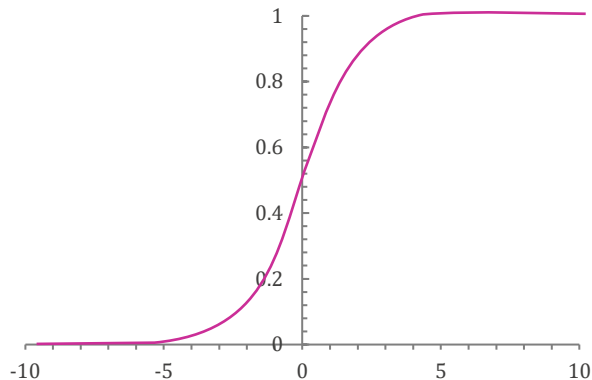
$$L_i = -\log\left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}}\right)$$

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

Summarising how Neural Networks Learn Continued



Sigmoid Activation



Sigmoid

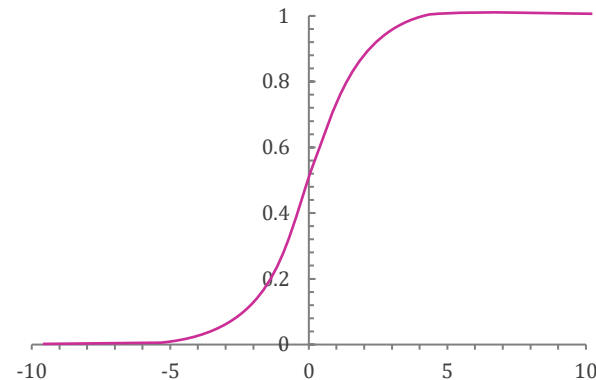
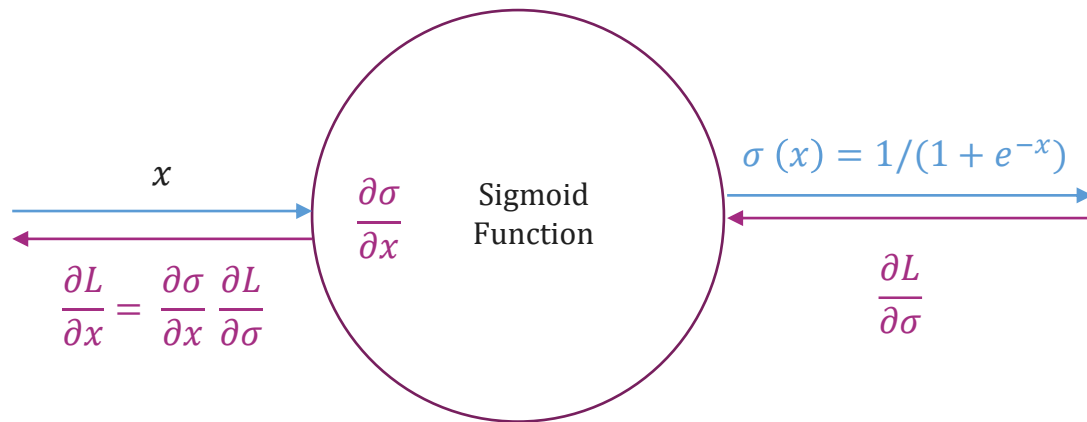
$$\sigma(x) = 1/(1 + e^{-x})$$

- Squashes outputs to range [0,1]
- Historically popular since they have nice interpretation as a saturated “firing rate” of a neuron

Problems:

- Saturated neurons “kill” the gradients
- Sigmoid outputs are not zero-centered
- Exponential is compute expensive

Sigmoid Activation



What happens when $x = -10$?

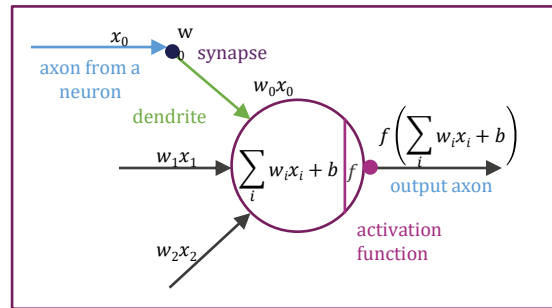
What happens when $x = 0$?

What happens when $x = 10$?

Consider what happens when the input to a neuron is always positive

$$y = f(\sum_i w_i x_i + b)$$

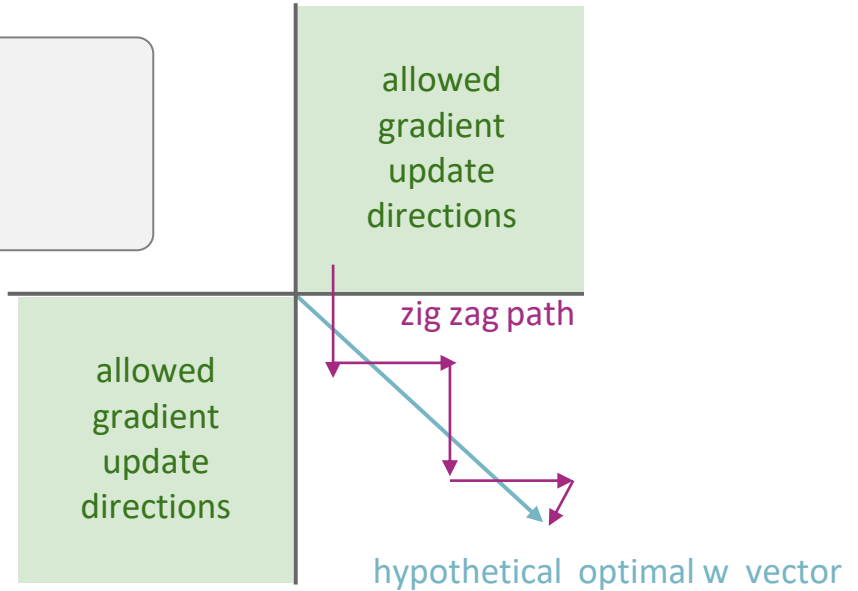
- **What can we say about the gradients on w ?**
 - We know that local gradient of sigmoid is always positive
 - We are assuming x is always positive
 - So, the sign of gradient for all w_i is the same as the sign of upstream scalar gradient.



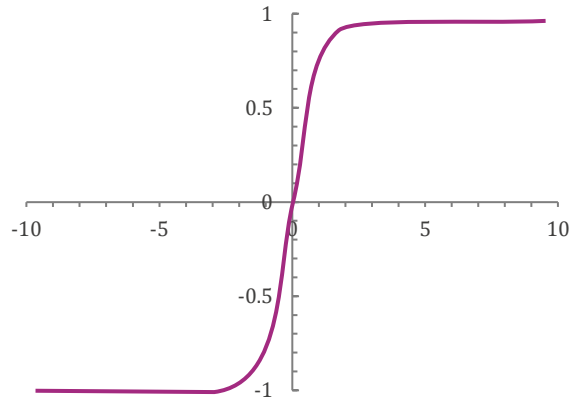
$$\frac{\partial L}{\partial w} = \sigma(\sum_i w_i x_i + b) (1 - \sigma(\sum_i w_i x_i + b)) x \text{ upstream gradient}$$

Consider what happens when the input to a neuron is always positive

- What can we say about the gradients on $\mathbf{w}$?
- **Always all positive or all negative**
(also why we would want zero-mean data)



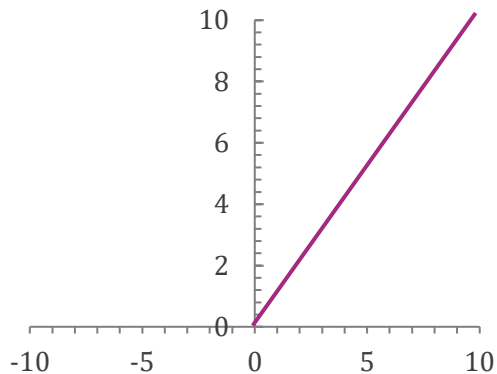
Tanh Activation



$\tanh(x)$

- Squashes output to range $[-1,1]$
- Zero centered
- Would kill gradients when saturated

ReLU Activation

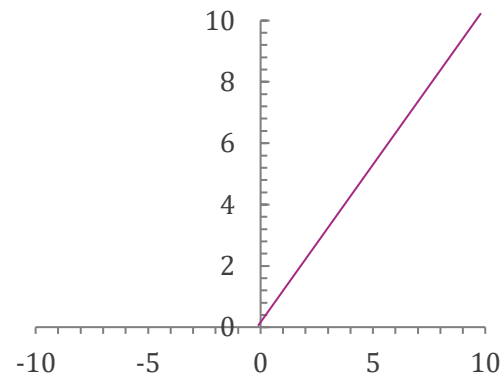
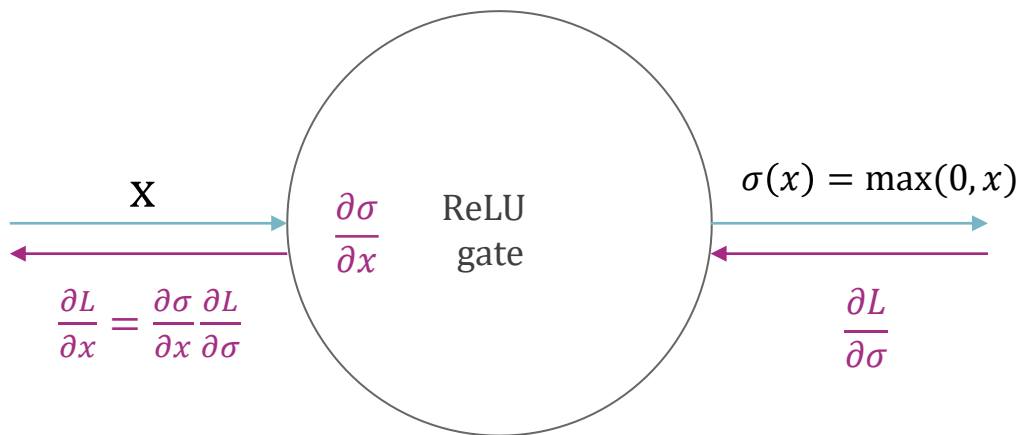


ReLU
(Rectified Linear Unit)

Computes $f(x) = \max(0, x)$

- Does not saturate (in positive region)
- Very computationally efficient
- Converges much faster than sigmoid/tanh in practice
- Not zero centered output

ReLU Activation



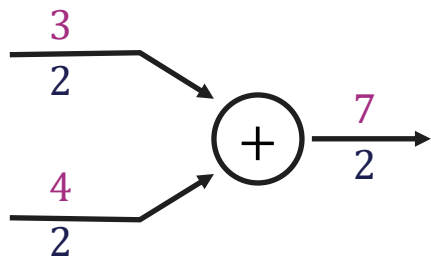
When $x = 10$?

When $x = 0$?

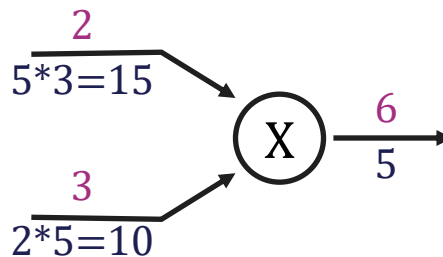
When $x = -10$?

Patterns in gradient flow

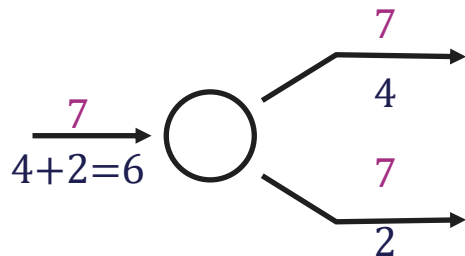
Add gate : gradient distributor



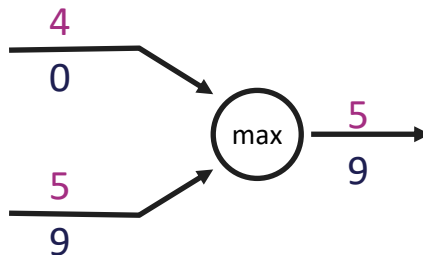
Multiply gate : swap multiplier



Copy gate : gradient adder

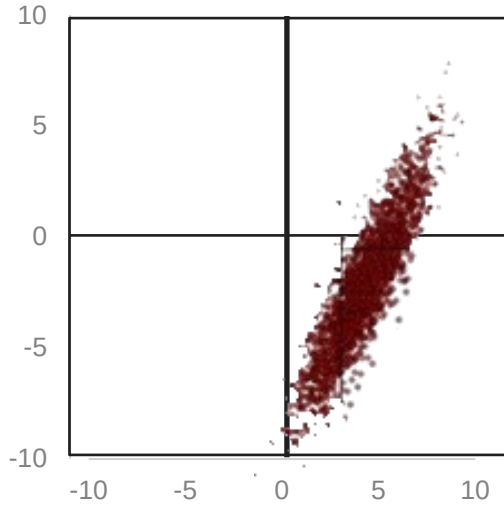


Max gate : gradient router

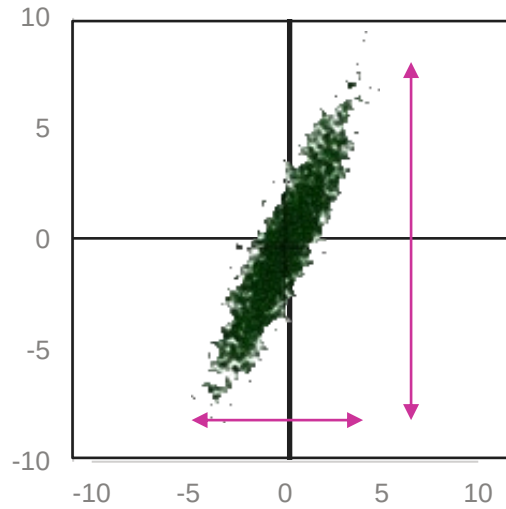


Pre-process the data

Original Data

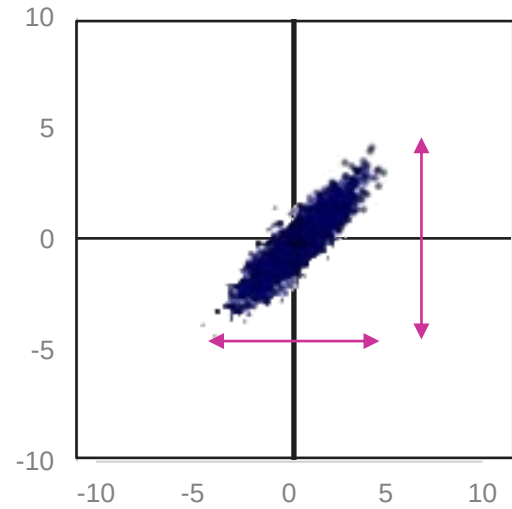


Zero-centered data



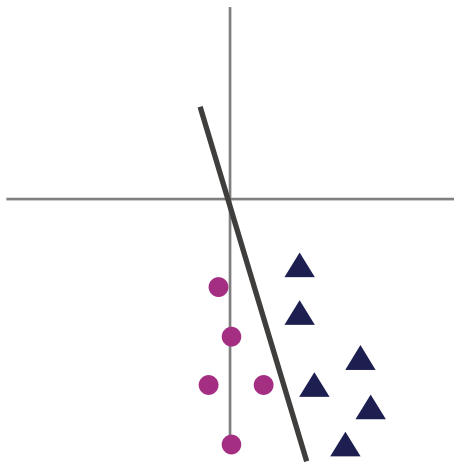
`X -= np.mean(X, axis = 0)`

Normalized data

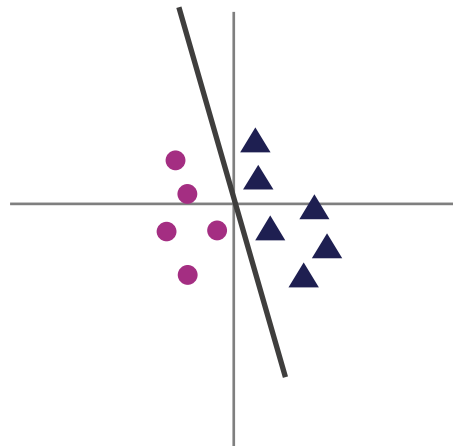


`X /= np.std(X, axis = 0)`

Before normalization : classification loss very sensitive to changes in weight matrix, Hard to optimize

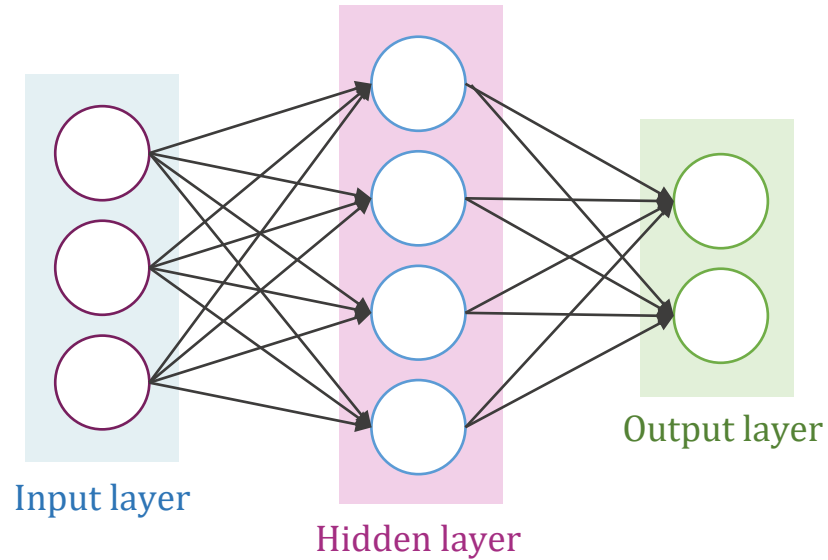


After normalization : less sensitive to small Changes in weights; easier to optimize



Weight Initialization

What happens when $W=0$ and no bias terms?



- First idea: **Small random numbers**
(Gaussian with zero mean and 0.01 standard deviation)

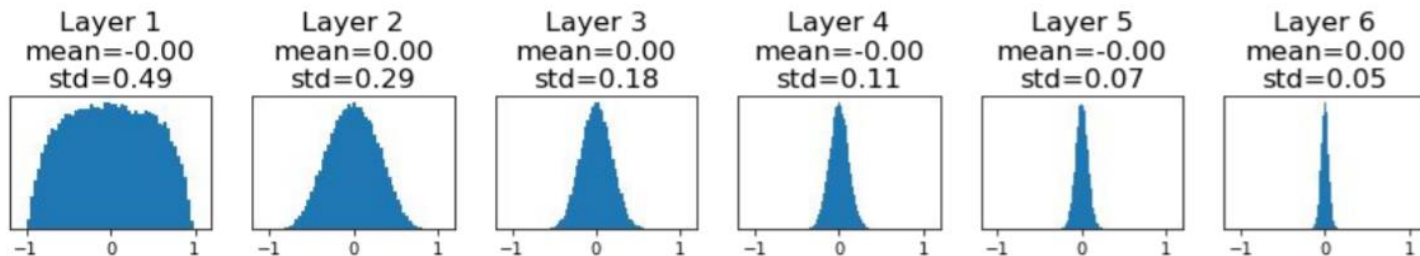
$$W = 0.01 * \text{np.random.randn}(D, H)$$

- Works fine for small networks, but can lead to non-homogeneous distributions of activations across the layers of a network.

Weight Initialization: Activation statistics

```
dims = [4096] * 7    Forward pass for a 6-layer
hs = []              net with hidden size 4096
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = 0.01 * np.random.randn(Din, Dout)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

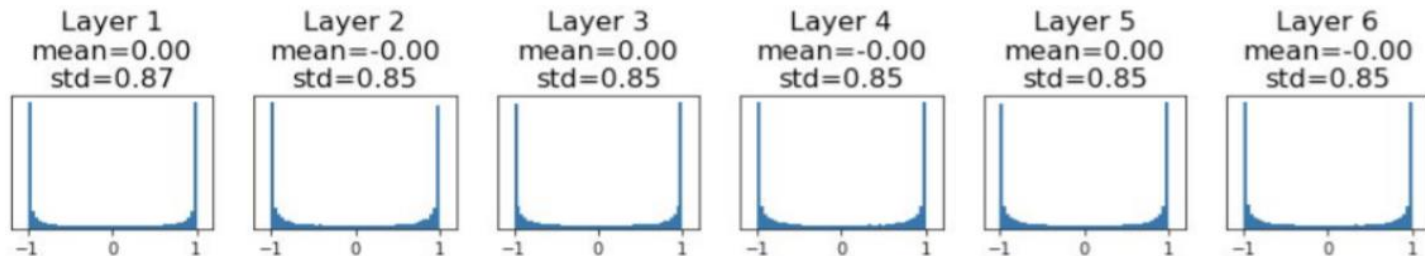
All activations tend to zero for deeper network layers.



Weight Initialization: Activation statistics

```
dims = [4096] * 7    Increase std of initial
hs = []              weights from 0.01 to 0.05
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = 0.05 * np.random.randn(Din, Dout)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

All activations tend to saturate.



Weight Initialization: Xavier Initialization

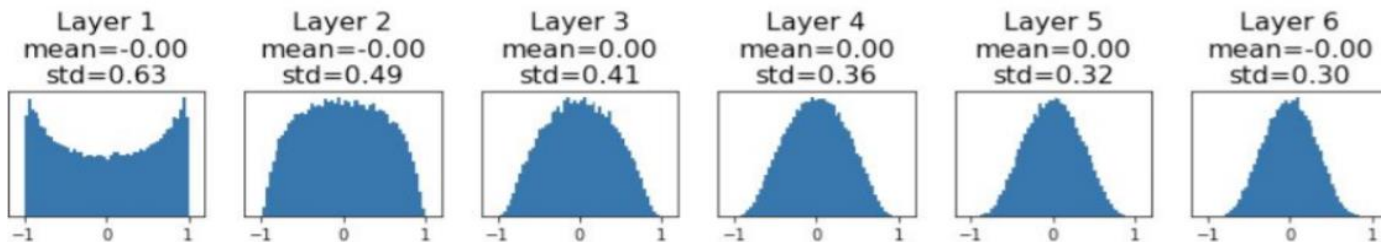
```

dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) / np.sqrt(Din)
    x = np.tanh(x.dot(W))
    hs.append(x)
    
```

“Xavier” initialization:

$\text{std} = 1/\sqrt{\text{Din}}$

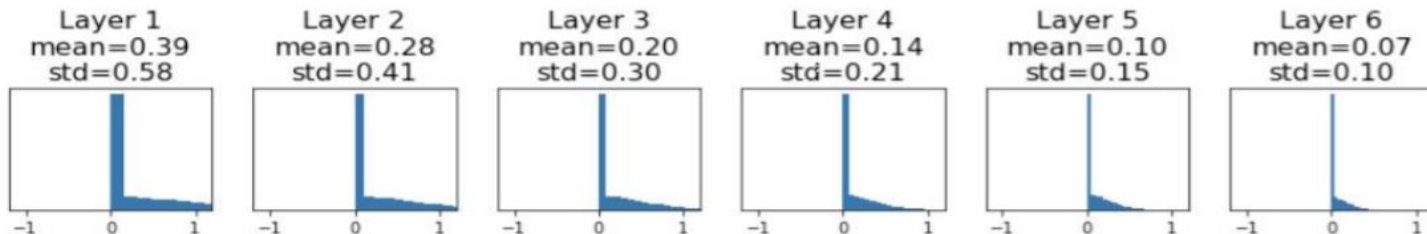
Activations are appropriately scaled for all layers.



Weight Initialization: Xavier Initialization what happens to ReLU

```
dims = [4096] * 7      Change from tanh to ReLU
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) / np.sqrt(Din)
    x = np.maximum(0, x.dot(W))
    hs.append(x)
```

Activations collapse to zero and thus, no learning.

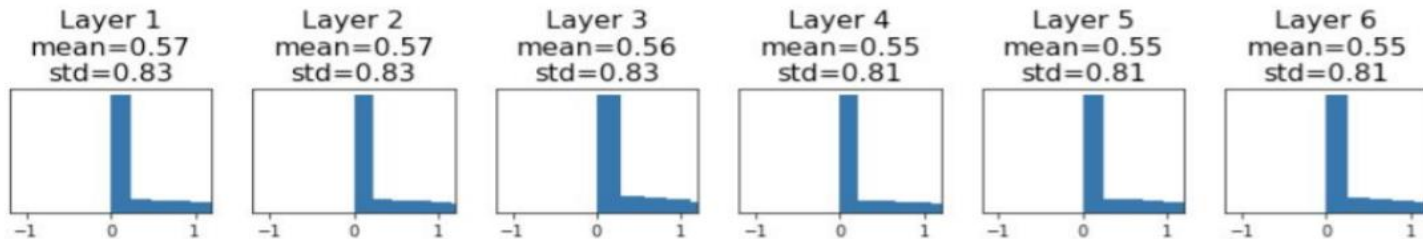


Weight Initialization: Kaiming / MSRA Initialization

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) * np.sqrt(2/Din)
    x = np.maximum(0, x.dot(W))
    hs.append(x)
```

ReLU correction: $\text{std} = \sqrt{2 / D_{\text{in}}}$

Activations are scaled just right for all layers.



He et al, "Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification", ICCV 2015

Consider a batch of activations at some layer, To make each dimension zero-mean unit-variance apply:

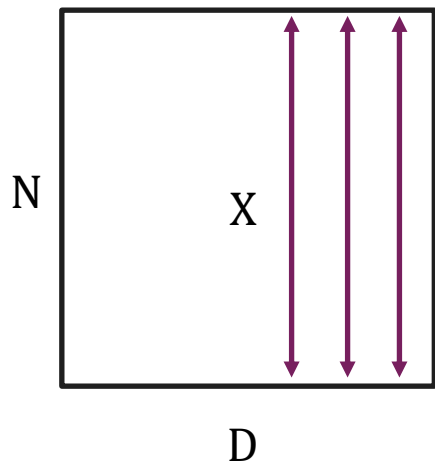
$$\hat{x}^{(k)} = \frac{x^{(k)} - E[x^{(k)}]}{\sqrt{\text{Var}[x^{(k)}]}}$$

“If you want zero-mean unit-variance activations? Just make them so.”

[Ioffe and Szegedy, 2015]

[Ioffe and Szegedy, 2015]

Input : $x : N \times D$



$$\mu_j = \frac{1}{N} \sum_{i=1}^N x_{i,j}$$

Per-channel mean,
Shape is D

$$\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (x_{i,j} - \mu_j)^2$$

Per-channel var,
shape is D

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}$$

Normalized x ,
Shape is $N \times D$

[Ioffe and Szegedy, 2015]

Input: $x : N \times D$

$$\mu_j = \frac{1}{N} \sum_{i=1}^N x_{i,j}$$

Per-channel mean,
shape is D

Learnable scale and shift parameters:

$$\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (x_{i,j} - \mu_j)^2$$

Per-channel variance,
shape is D

$\gamma, \beta : D$

Learning $\gamma = \sigma$
 $\beta = \mu$ will recover the
identity function

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}$$

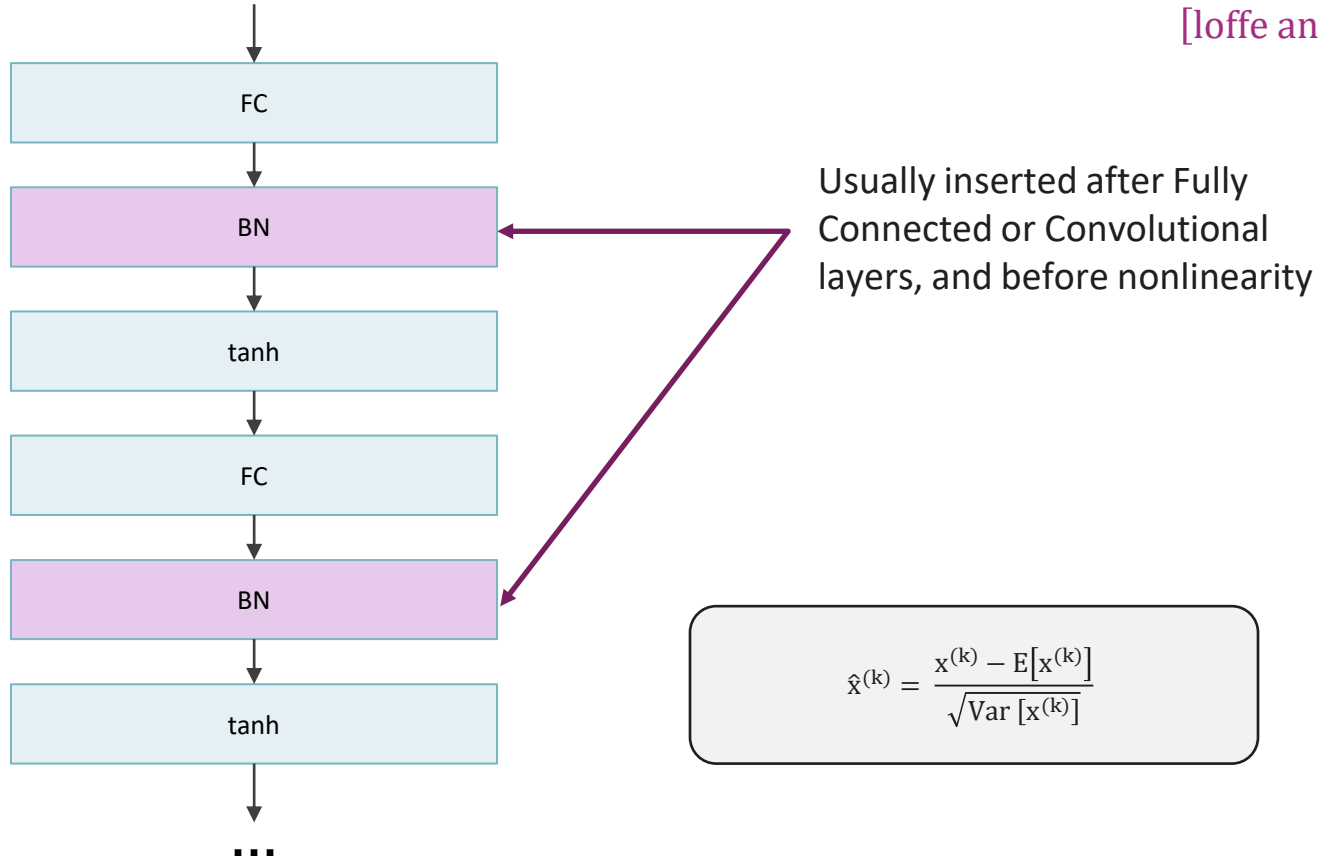
Normalized x,
Shape is N x D

$$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j$$

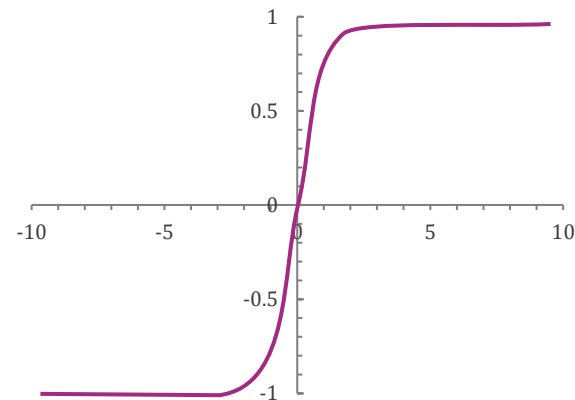
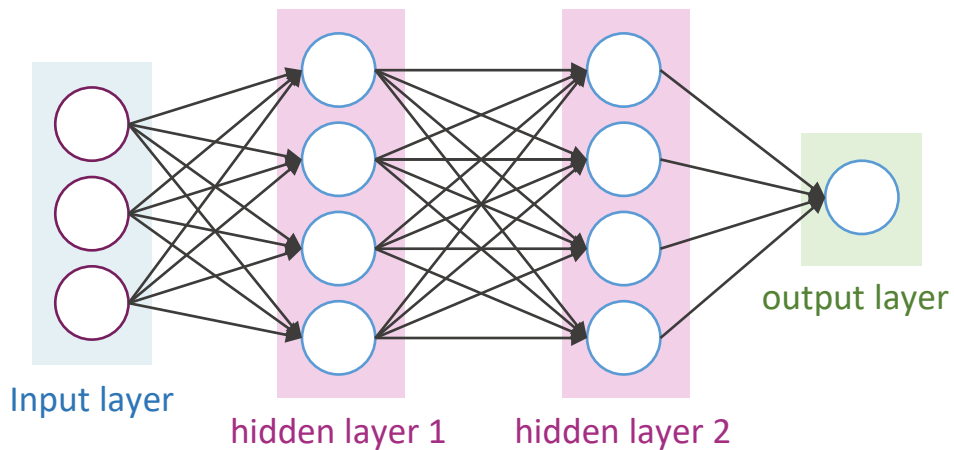
Output
Shape is N x D

Understanding Batch Normalization

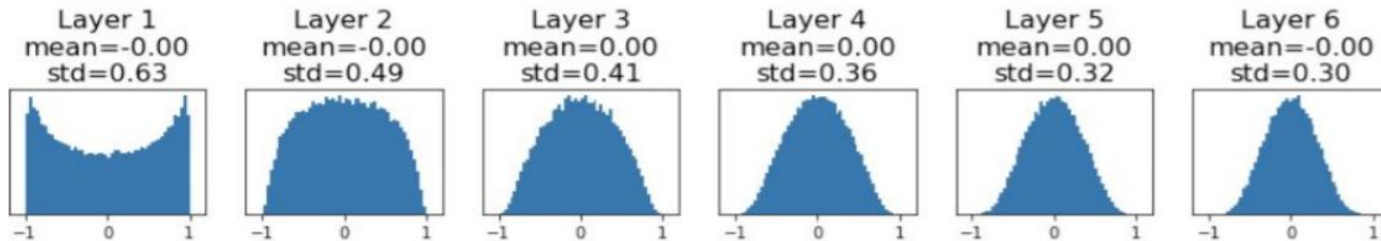
[Ioffe and Szegedy, 2015]



Understanding Batch Normalization



$\tanh(x)$



Batch Normalization : Test-Time

Estimates depend on batch, so we can't do this at test-time

Input : $x : N \times D$

$\mu_j =$ (Running) average of
Values seen during training

**Per-channel mean,
Shape is D**

**Learnable Scale and
shift parameters :**

$\sigma_j^2 =$ (Running) average of
Values seen during training

**Per-channel var,
Shape is D**

$\gamma, \beta : D$

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

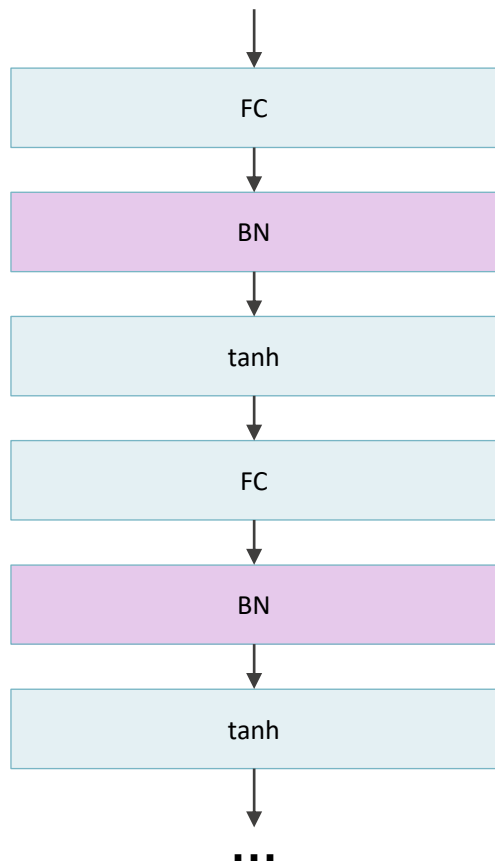
**Normalized x,
Shape is N x D**

$$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j$$

**Output,
Shape is N x D**

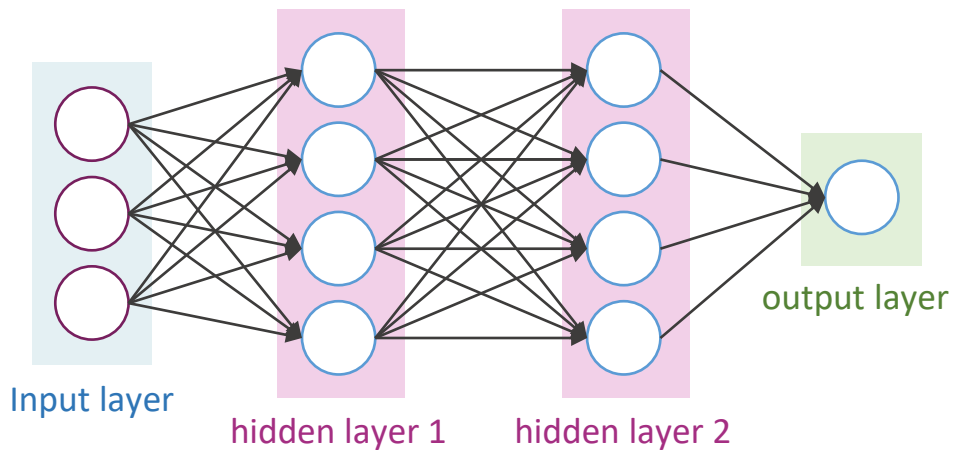
Batch Normalization

[Ioffe and Szegedy, 2015]



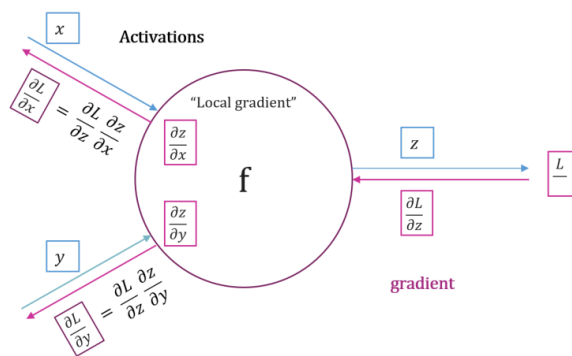
- Makes deep networks **much** easier to train!
- Improves gradient flow
- Allows higher learning rates, faster convergence
- Networks become more robust to initialization
- Acts as regularization during training
- Zero overhead at test-time
- Behaves differently during training and testing

Optimizing the network parameters



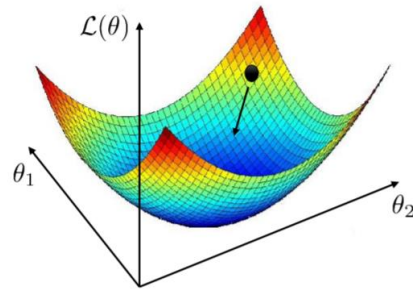
Gradient

$$\nabla_{\theta} \mathcal{L}(\theta) = \begin{bmatrix} \frac{d\mathcal{L}(\theta)}{d\theta_1} \\ \frac{d\mathcal{L}(\theta)}{d\theta_2} \\ \vdots \\ \frac{d\mathcal{L}(\theta)}{d\theta_n} \end{bmatrix}$$

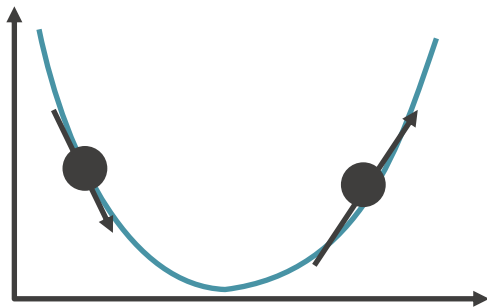


Gradient descent and Loss Function Landscape

1. Find a direction v where $\mathcal{L}(\theta)$ decreases
2. $\theta \leftarrow \theta + av$
3. $\theta_i \leftarrow \theta_i - a \frac{d\mathcal{L}(\theta)}{d\theta_i}$ a is some small constant called "learning rate" or "step size"



Which way does $\mathcal{L}(\theta)$ decrease?



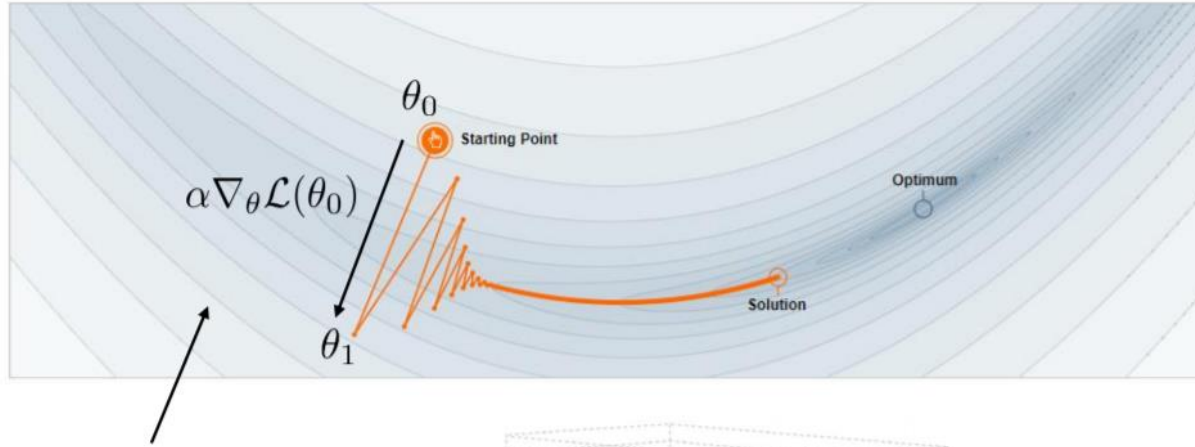
- Negative slope = go to the right
- Positive slope = go to the left
- In general :
 - For each dimension, go in the direction opposite the slope **along that dimension**

gradient :

$$\nabla_{\theta} \mathcal{L}(\theta) = \begin{bmatrix} \frac{d\mathcal{L}(\theta)}{d\theta_1} \\ \frac{d\mathcal{L}(\theta)}{d\theta_2} \\ \vdots \\ \frac{d\mathcal{L}(\theta)}{d\theta_n} \end{bmatrix}$$

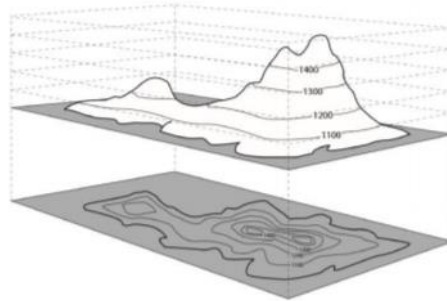
$$v_1 = - \frac{d\mathcal{L}(\theta)}{d\theta_1} \quad v_2 = - \frac{d\mathcal{L}(\theta)}{d\theta_2} \text{ etc.}$$

Visualizing gradient descent

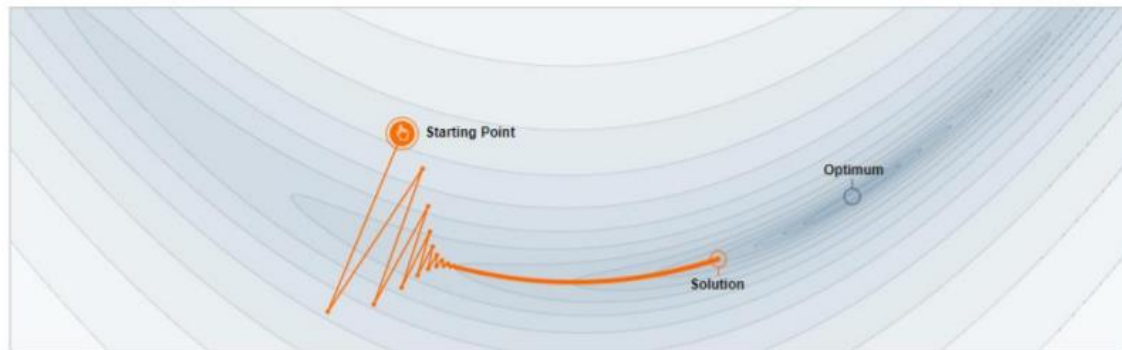


Level set contours

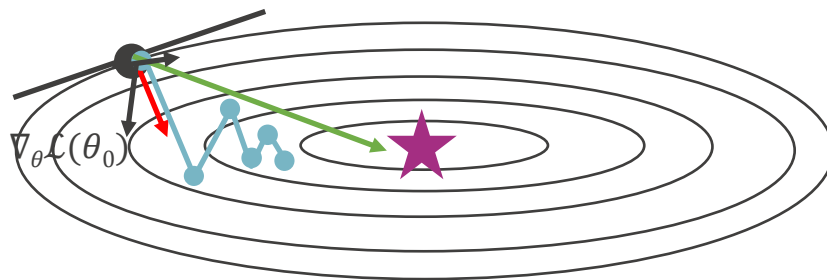
For all θ values along a line
 $\mathcal{L}(\theta)$ takes on the same value



What's going on?

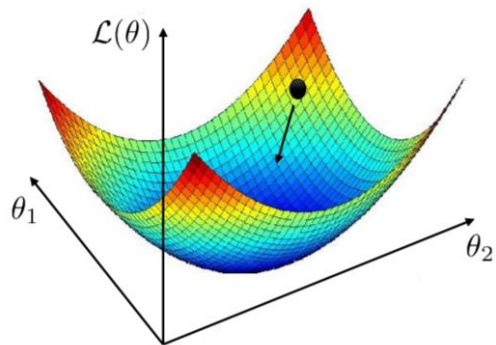


We don't always move toward the optimum, instead towards the direction of steepest descent.

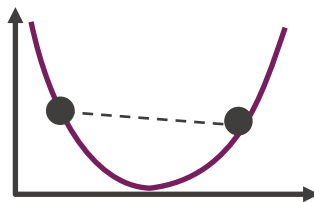


$$\theta_i \leftarrow \theta_i - a \frac{d\mathcal{L}(\theta)}{d\theta_i}$$

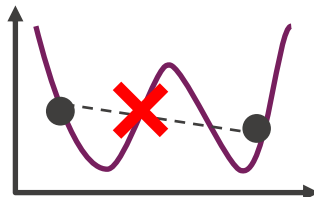
The steepest direction is not always best!



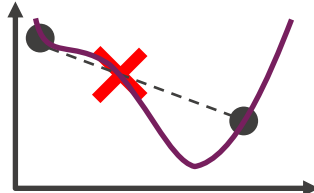
Convexity :



Non Convexity :

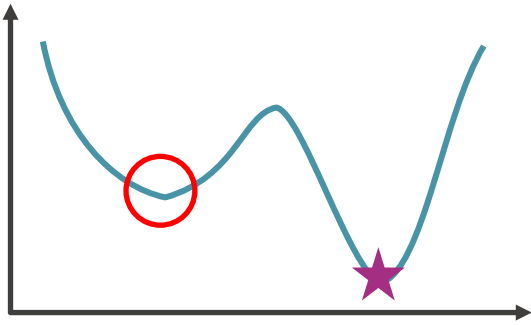


Non Convexity :

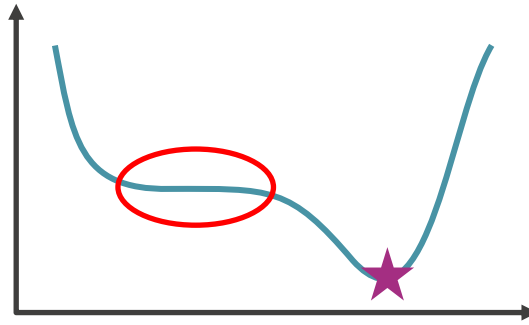


- A function is convex if a line segment between any two points lies entirely “above” the graph
- Convex functions are desirable because simple algorithms like gradient descent have strong guarantees around convergence.
- While it **doesn't** mean that gradient descent works well for all convex functions

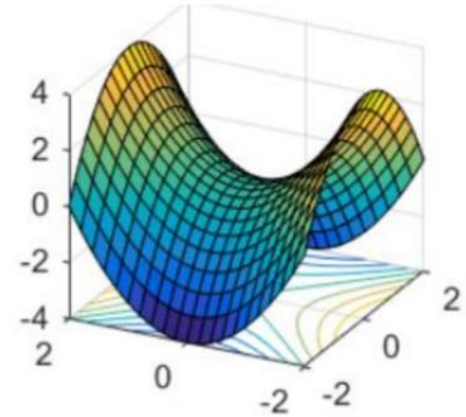
The Geography of a Loss Landscape in Deep Learning



The local optimum

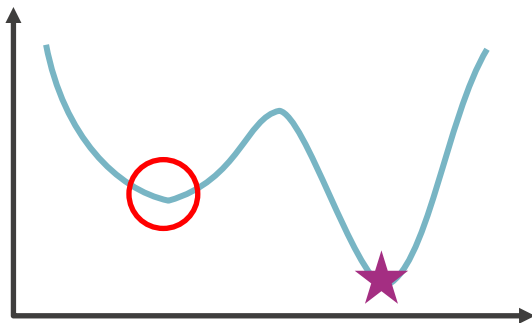


The plateau



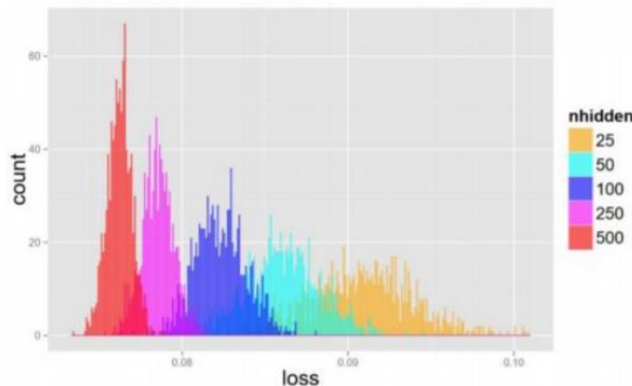
The saddle point

Local optima

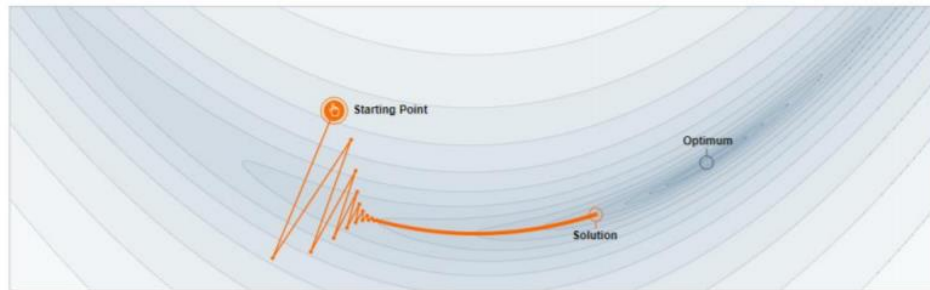
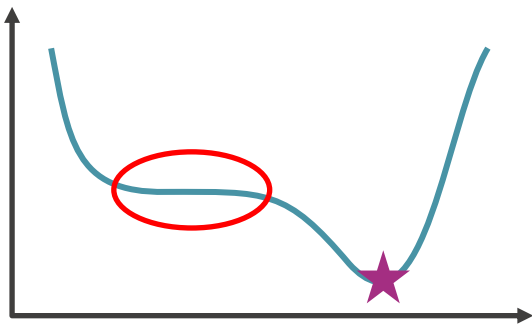


- The most obvious issue with non-convex loss landscapes
- One of the big reasons people used to worry about neural networks, since gradient descent could converge to a solution that is arbitrarily worse than the global optimum.

- A bit surprisingly, this becomes less of an issue as the number of parameters increases!
- For big networks, local optima exist, but tend to be not much worse than global optima.

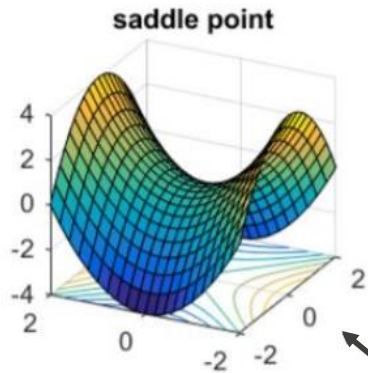


Choromanska, Henaff, Mathieu, Ben Arous, LeCun.
The Loss Surface of Multilayer Networks.



- Can't just choose tiny learning rates to prevent oscillation!
- Need learning rates to be large enough not to get stuck in a plateau
- We'll learn about **Momentum**, which helps in this case

Saddle Points



Critical points :

Any point where $\nabla_{\theta} \mathcal{L}(\theta) = 0$
Is it a **maximum**, **minimum**,
or **saddle** ?

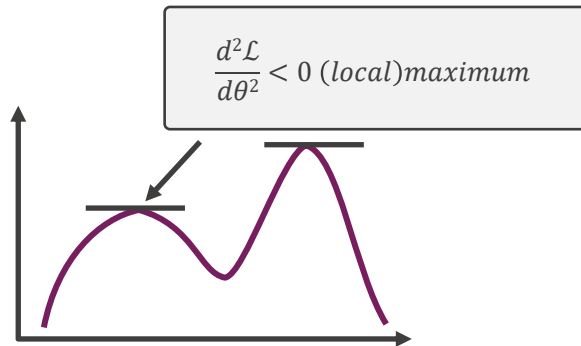
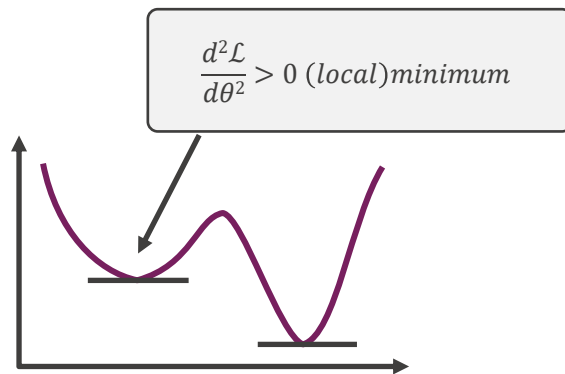
In 2 dimensions :

$$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

Hessian matrix:

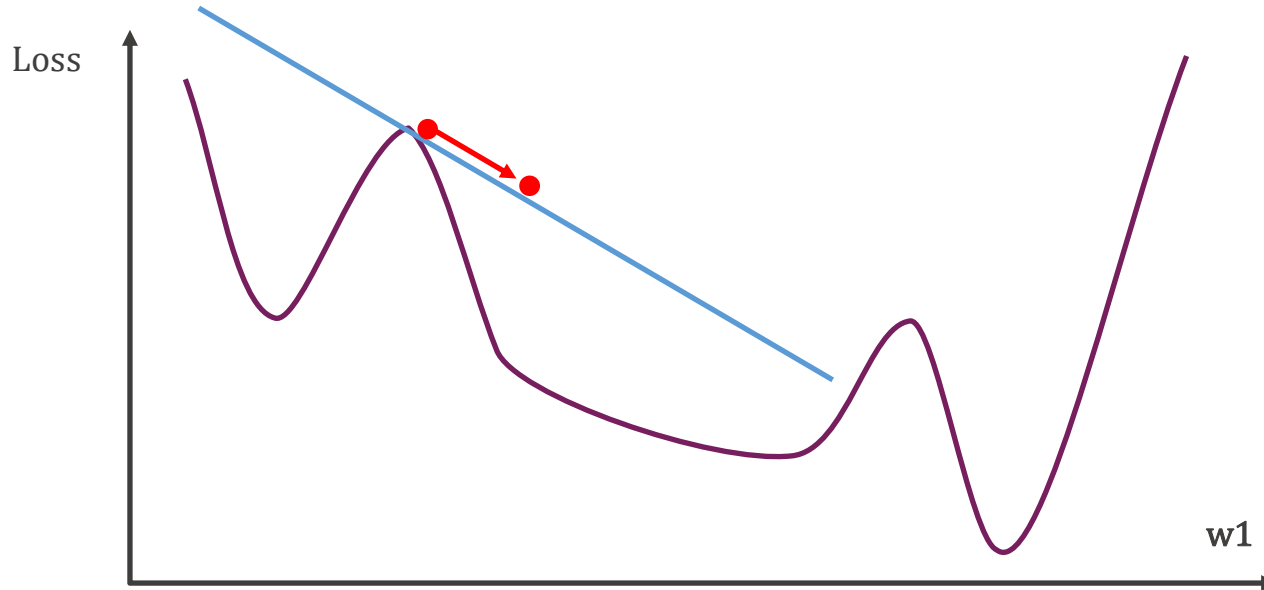
$$\begin{bmatrix} \frac{d^2 \mathcal{L}}{d\theta_1 d\theta_1} & \frac{d^2 \mathcal{L}}{d\theta_1 d\theta_2} & \frac{d^2 \mathcal{L}}{d\theta_1 d\theta_3} \\ \frac{d^2 \mathcal{L}}{d\theta_2 d\theta_1} & \frac{d^2 \mathcal{L}}{d\theta_2 d\theta_2} & \frac{d^2 \mathcal{L}}{d\theta_2 d\theta_3} \\ \frac{d^2 \mathcal{L}}{d\theta_3 d\theta_1} & \frac{d^2 \mathcal{L}}{d\theta_3 d\theta_2} & \frac{d^2 \mathcal{L}}{d\theta_3 d\theta_3} \end{bmatrix}$$

Only maximum or minimum if
diagonal entries are either all
positive or all negative.



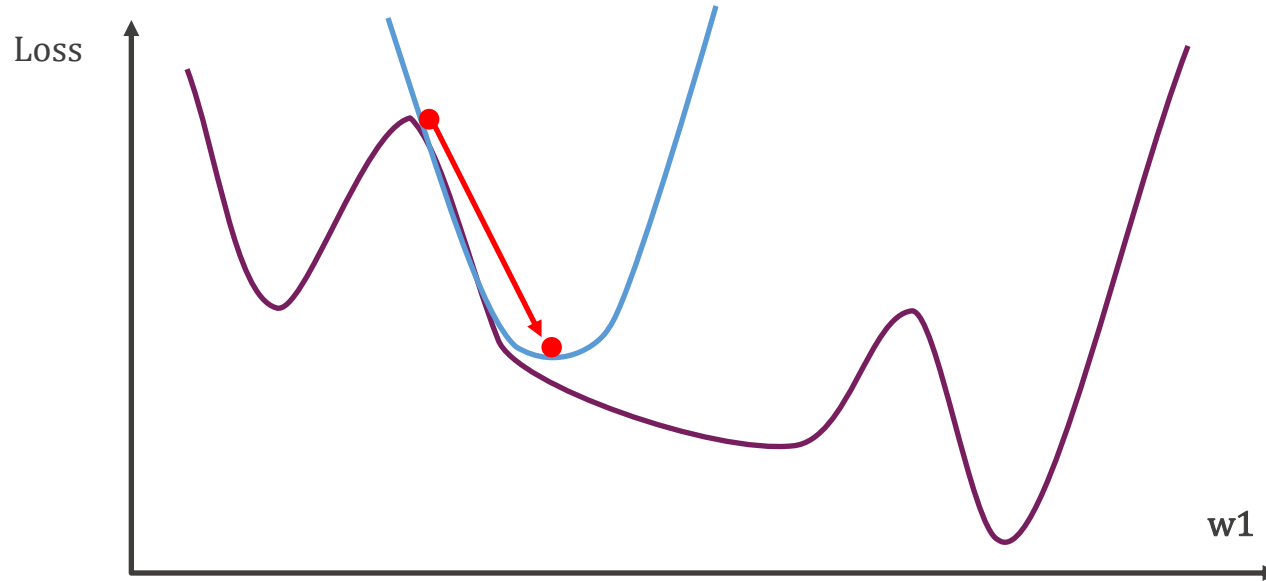
First-Order Optimization

1. Use gradient form linear approximation
2. Step to minimize the approximation



Second-Order Optimization

1. Use gradient **and Hessian** to form **quadratic** approximation
2. Step to the **minima** the approximation



Second-Order Optimization

- Second-order Taylor expansion :

$$J(\theta) \approx J(\theta_0) + (\theta - \theta_0)^T \nabla \theta J(\theta_0) + \frac{1}{2} (\theta - \theta_0)^T H (\theta - \theta_0)$$

- Solving for the critical point we obtain the Newton parameter update :

$$\theta^* = \theta_0 - H^{-1} \nabla \theta J(\theta_0)$$

Hessian has $O(N^2)$ elements
Inverting takes $O(N^3)$
 N =(Tens or Hundreds of) Millions

Example of Training with a Multiclass SVM Loss

Suppose: 3 training examples, 3 classes.
With some W the scores $f(x_i, W) = Wx$ are:



Cat	3.2	1.3	2.2
Car	5.1	4.9	2.5
Frog	-1.7	2.0	-3.1
Losses	2.9	0	10.9

Multiclass SVM loss

Given an example (x_i, y_i)

Where x_i is the image and

Where y_i is the (integer) label,

and using the shorthand for the scores vector: $s = f(x_i, W)$

The SVM loss has the form:

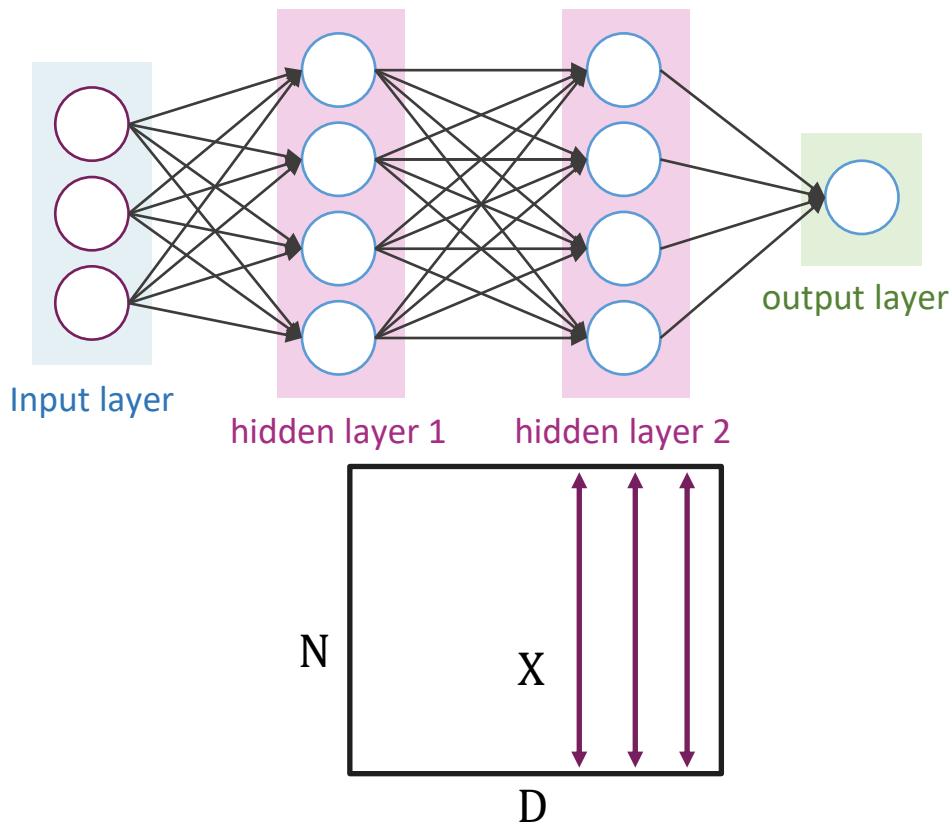
$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

and the full training loss is the mean over all examples in the training data:

$$L = \frac{1}{N} \sum_{i=1}^N L_i$$

$$L = (2.9 + 0 + 10.9)/3 \\ = 4.6$$

Gradient Descent in Practice: Motivating Stochastic Gradient Descent



Gradient

$$\nabla_{\theta} \mathcal{L}(\theta) = \begin{bmatrix} \frac{d\mathcal{L}(\theta)}{d\theta_1} \\ \frac{d\mathcal{L}(\theta)}{d\theta_2} \\ \vdots \\ \frac{d\mathcal{L}(\theta)}{d\theta_n} \end{bmatrix}$$

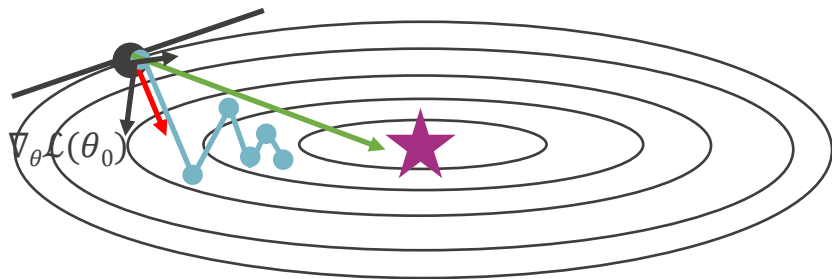
$$\theta_i \leftarrow \theta_i - a \frac{d\mathcal{L}(\theta)}{d\theta_i}$$

SGD + Momentum

SGD

$$\theta_{t+1} = \theta_t - \alpha \nabla f(\theta_t)$$

```
while True:
    dx = compute_gradient(x)
    x -= learning_rate * dx
```



SGD + Momentum

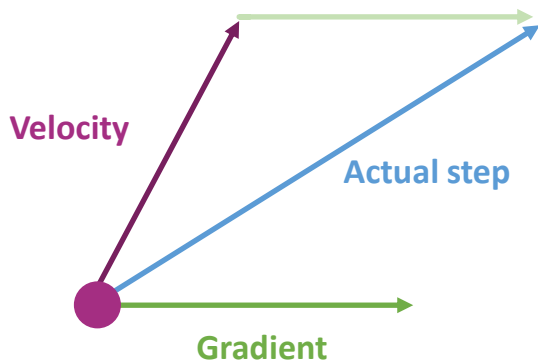
$$\begin{aligned} v_{t+1} &= \rho v_t + \nabla f(\theta_t) \\ \theta_{t+1} &= \theta_t - \alpha v_{t+1} \end{aligned}$$

```
vx = 0
while True:
    dx = compute_gradient(x)
    vx = rho * vx + dx
    x -= learning_rate * vx
```

- Builds up “velocity” as a running mean of gradients
- rho gives “friction”; typically rho is set to value greater than 0.5

Nesterov Momentum

Momentum update:



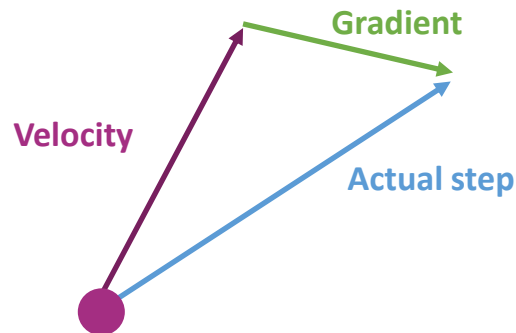
Combine gradient at current point with Velocity to get step used to update weights

Nesterov, "A method of solving a convex programming problem with convergence rate $O(1/k^2)$ ", 1983

Nesterov, "Introductory lectures on convex optimization: a basic course", 2004

Sutskever et al, "On the importance of initialization and momentum in deep learning", ICML 2013

Nesterov Momentum



"Look ahead" to the point where updating using velocity would take us; compute gradient there and mix it with velocity to get actual update direction

AdaGrad and RMSProp

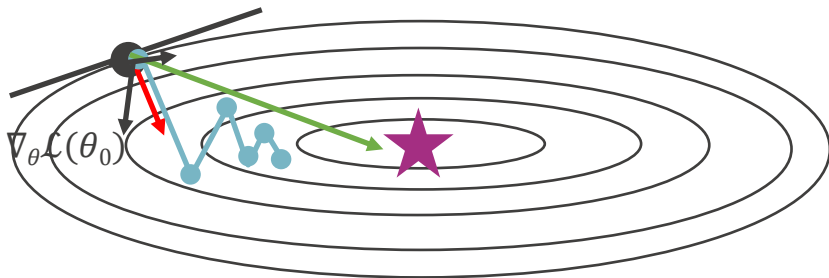
AdaGrad

```
grad_squared = 0
while True:
    dx = compute_gradient(x)
    grad_squared += dx * dx
    x -= learning_rate * dx / (np.sqrt(grad_squared) + 1e-7)
```



RMSProp

```
grad_squared = 0
while True:
    dx = compute_gradient(x)
    grad_squared = decay_rate * grad_squared + (1 - decay_rate) * dx * dx
    x -= learning_rate * dx / (np.sqrt(grad_squared) + 1e-7)
```



Adam: Combination of Momentum and RMSProp

```
first_moment = 0
second_moment = 0
for t in range(1, num_iterations):
    dx = compute_gradient(x)
    first_moment = beta1 * first_moment + (1 - beta1) * dx
    second_moment = beta2 * second_moment + (1 - beta2) * dx * dx
    first_unbias = first_moment / (1 - beta1 ** t)
    second_unbias = second_moment / (1 - beta2 ** t)
    x -= learning_rate * first_unbias / (np.sqrt(second_unbias) + 1e-7))
```

Momentum

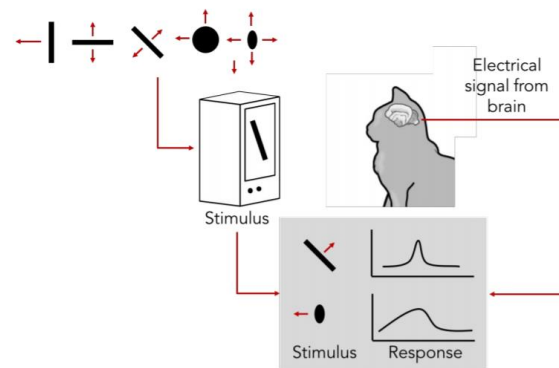
Bias correction

AdaGrad / RMSProp

Kingma and Ba, “**Adam: A method for stochastic optimization**”, ICLR 2015

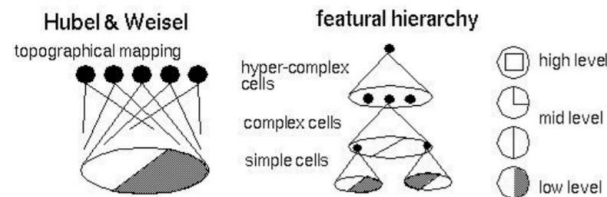
Hubel & Wiesel, 1959

Receptive Fields of Single
Neurons in the cat's striate cortex

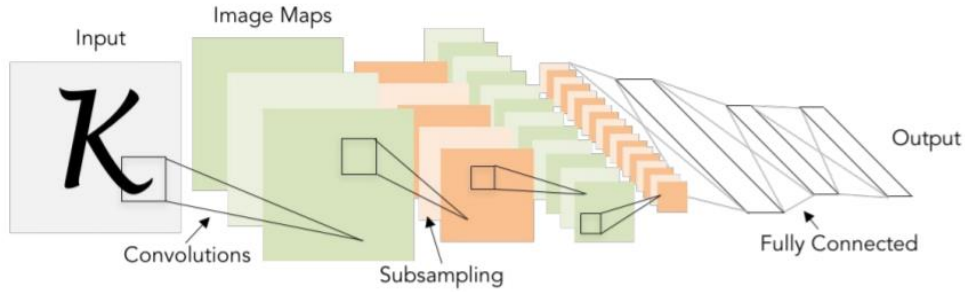


1962

Receptive Fields, Binocular
Interaction and Functional Architecture in the
cat's Visual Cortex

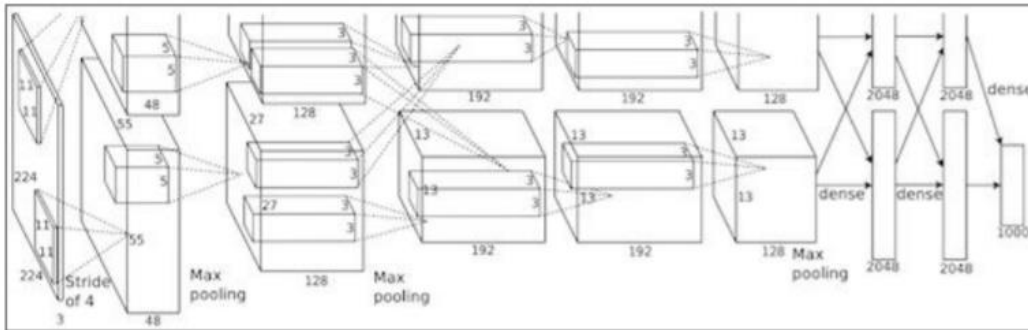


CNNs – A History



LeNet-5

Gradient-based learning applied to document recognition [LeCun, Bottou, Bengio, Haffner 1998]



AlexNet

ImageNet Classification with Deep Convolutional Neural Networks [Krizhevsky, Sutskever, Hinton, 2012]



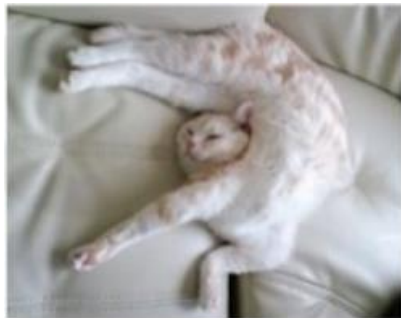
Challenges



Viewpoint variation



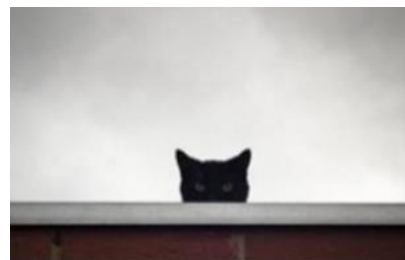
Varying conditions of illumination



Deformation



Camouflage

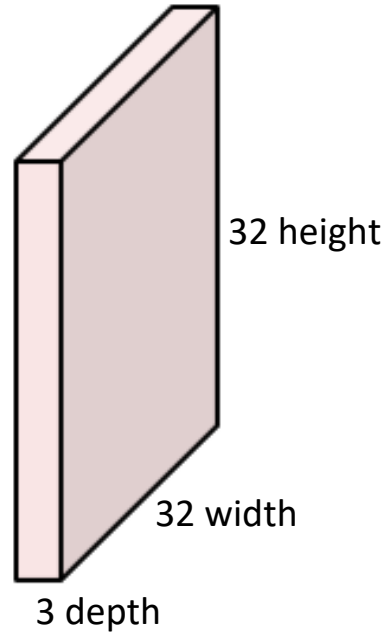


Occlusion



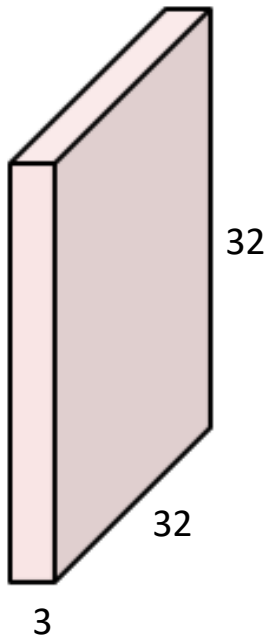
Intra-class variation

Preserve Spatial Structure 32 x 32 x 3 image



Convolution Layer

32 x 32 x 3 image

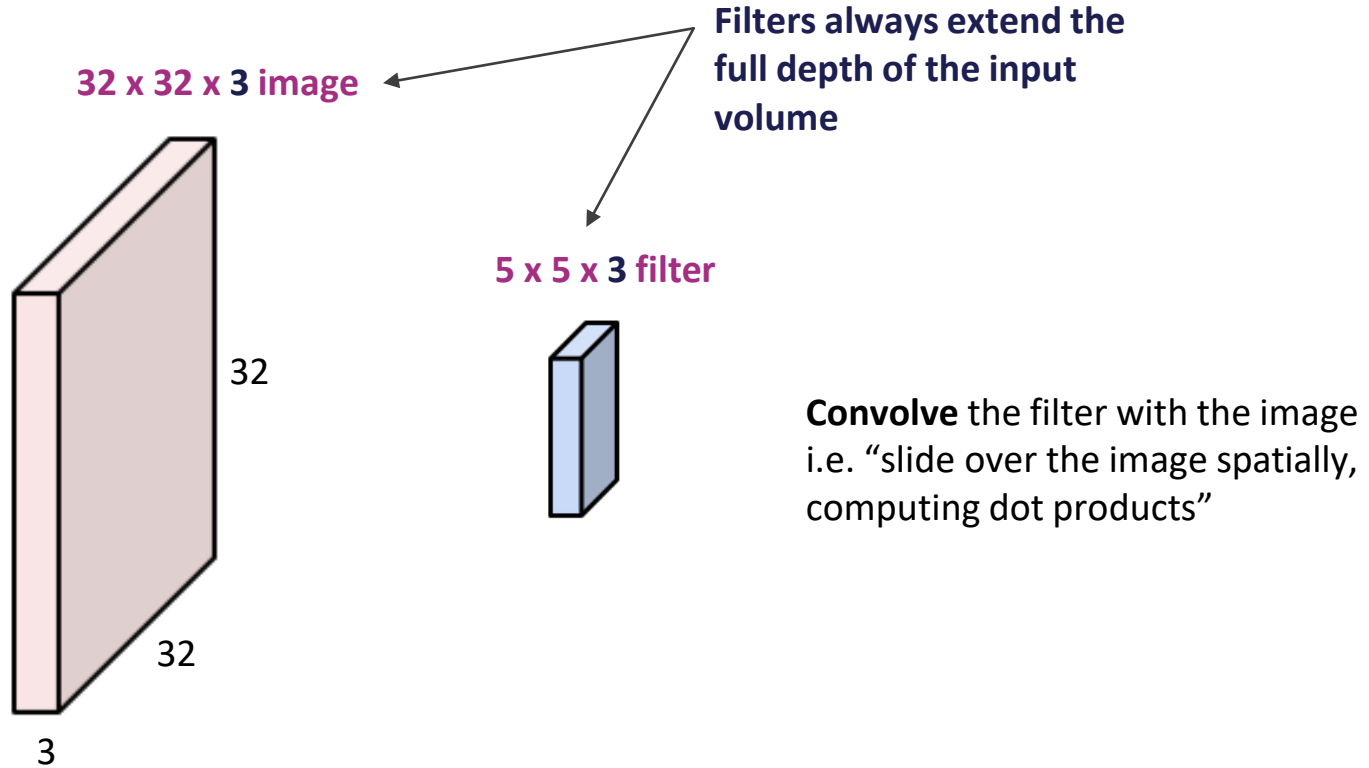


5 x 5 x 3 image

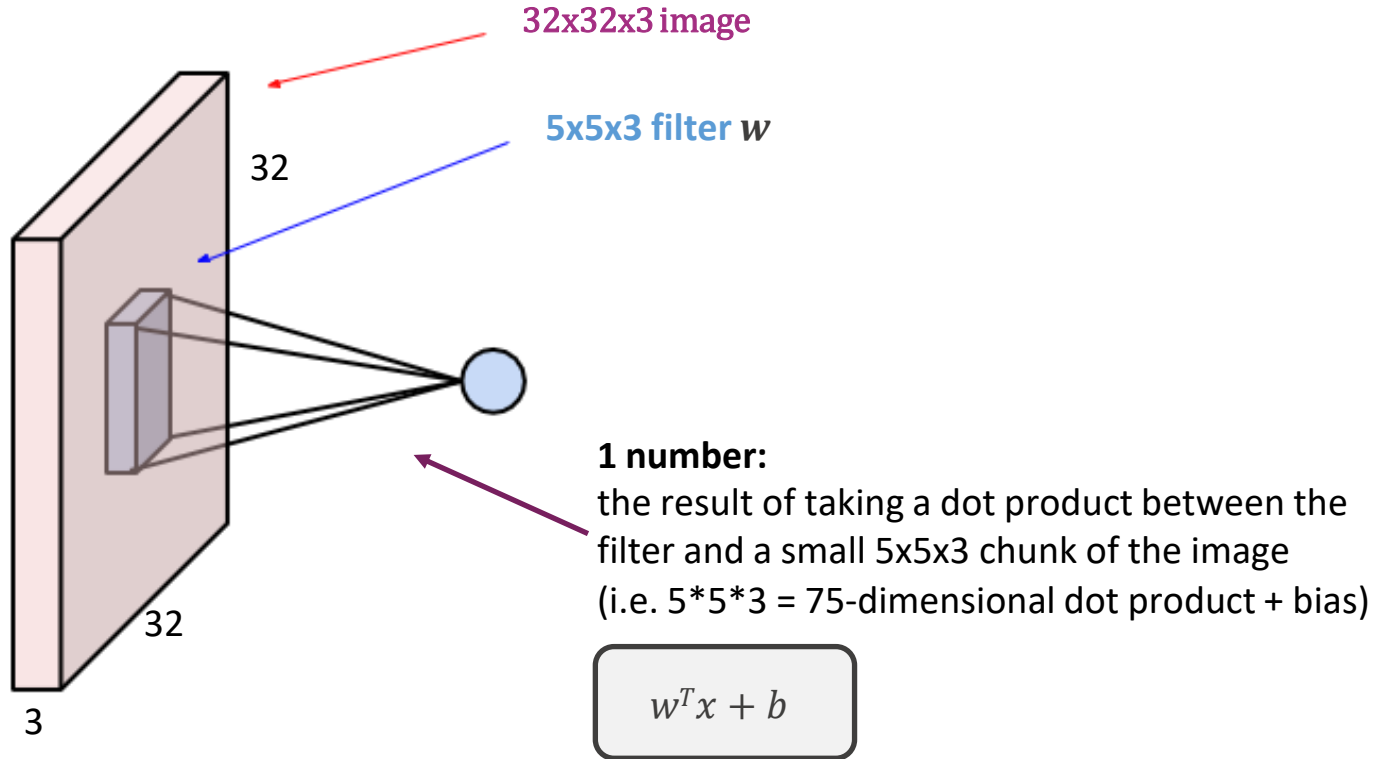


Convolve the filter with the image
i.e. “slide over the image spatially,
computing dot products”

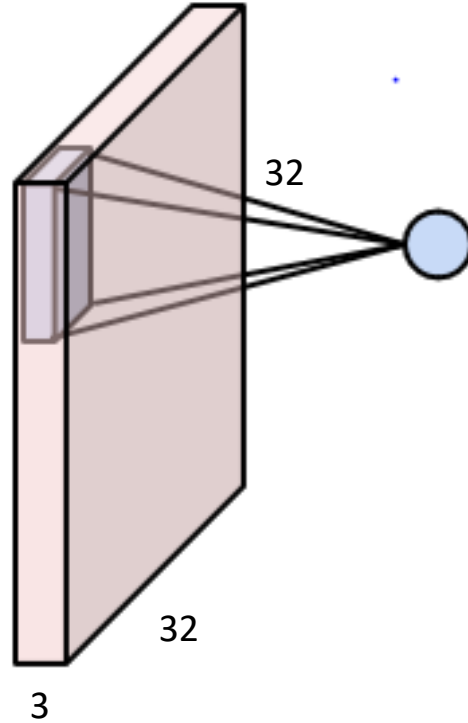
Convolution Layer



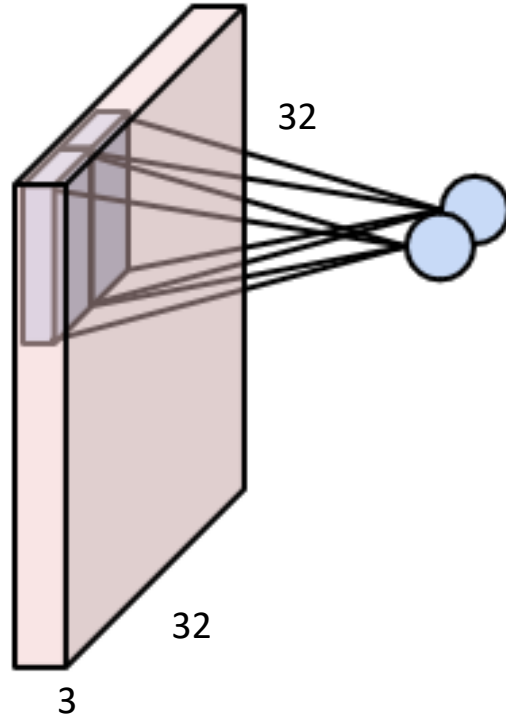
Convolution Layer



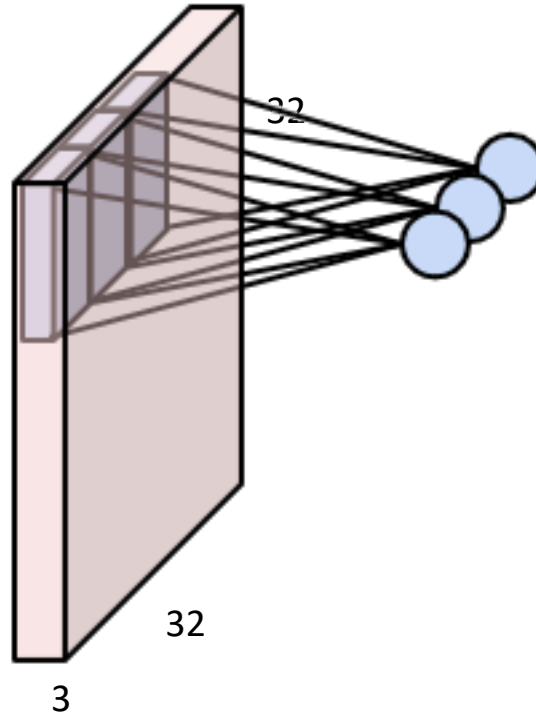
Convolution Layer



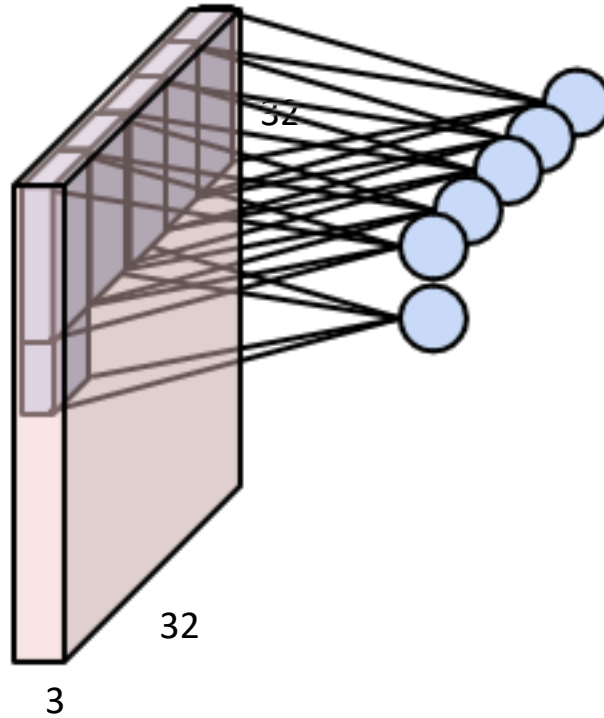
Convolution Layer



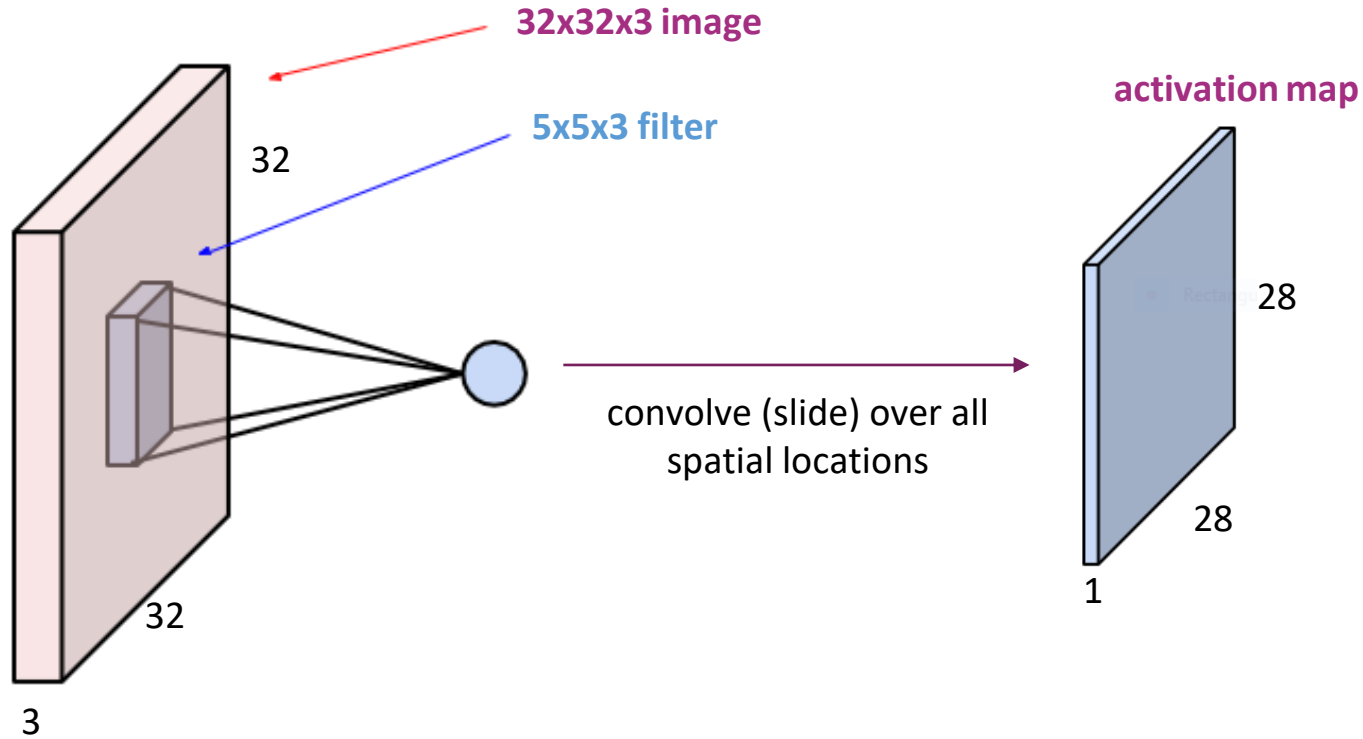
Convolution Layer



Convolution Layer

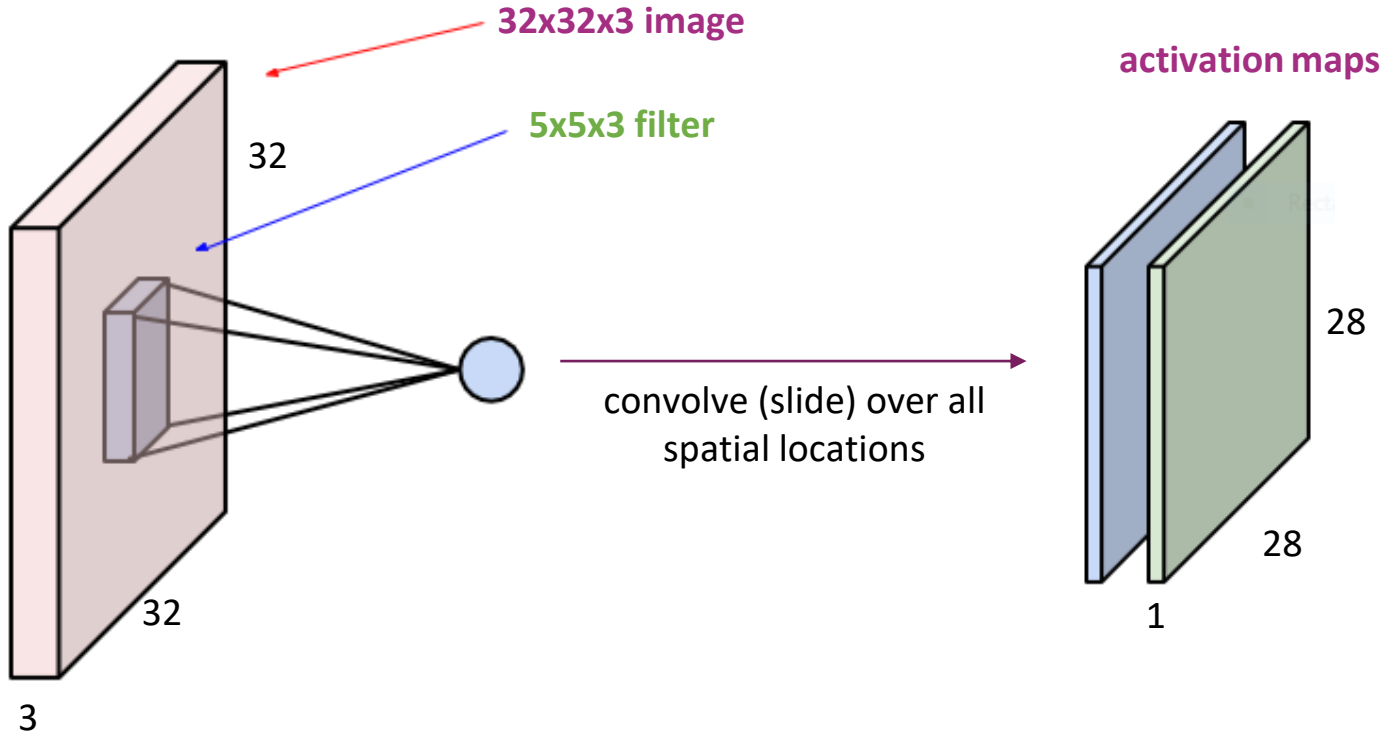


Convolution Layer

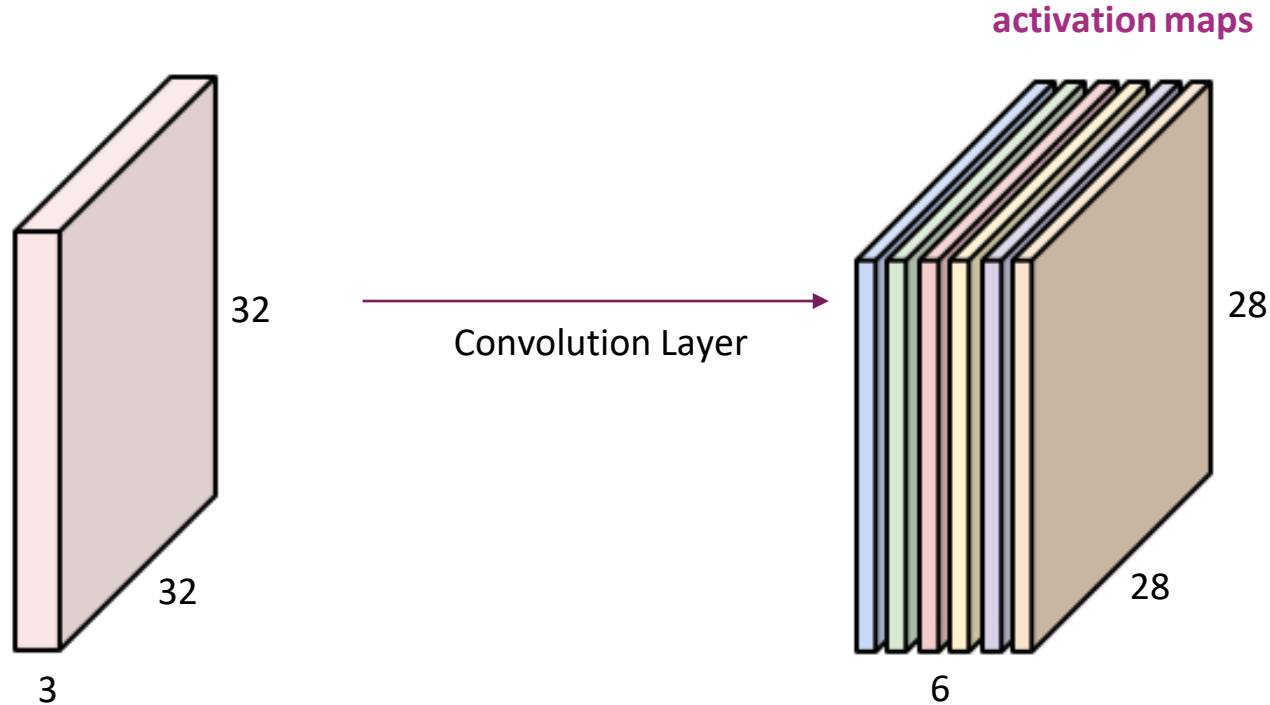


Convolution Layer

consider a second, **green** filter

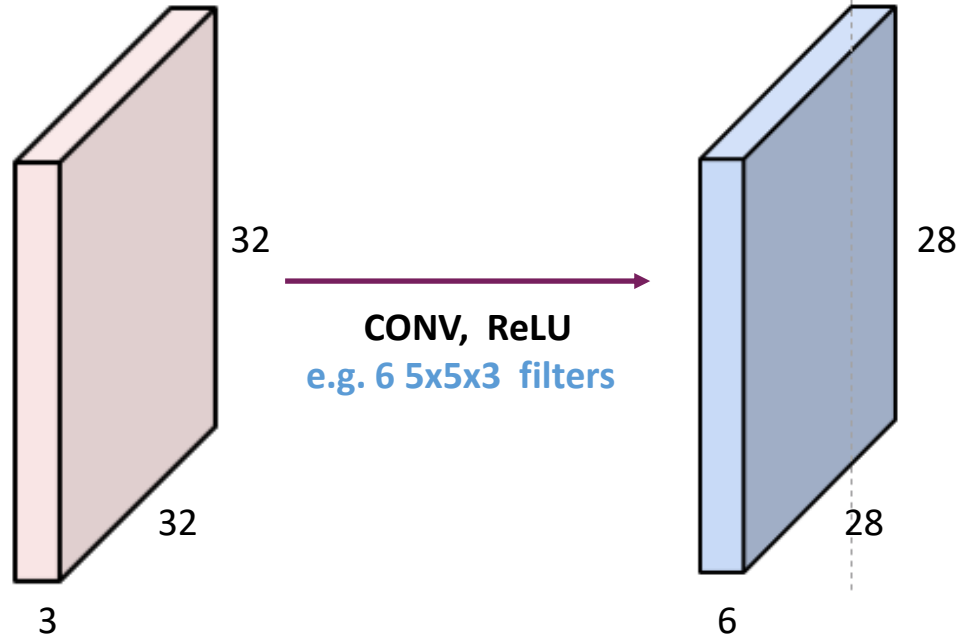


For example, if we had 6 5x5 filters, we'll get 6 separate activation maps:

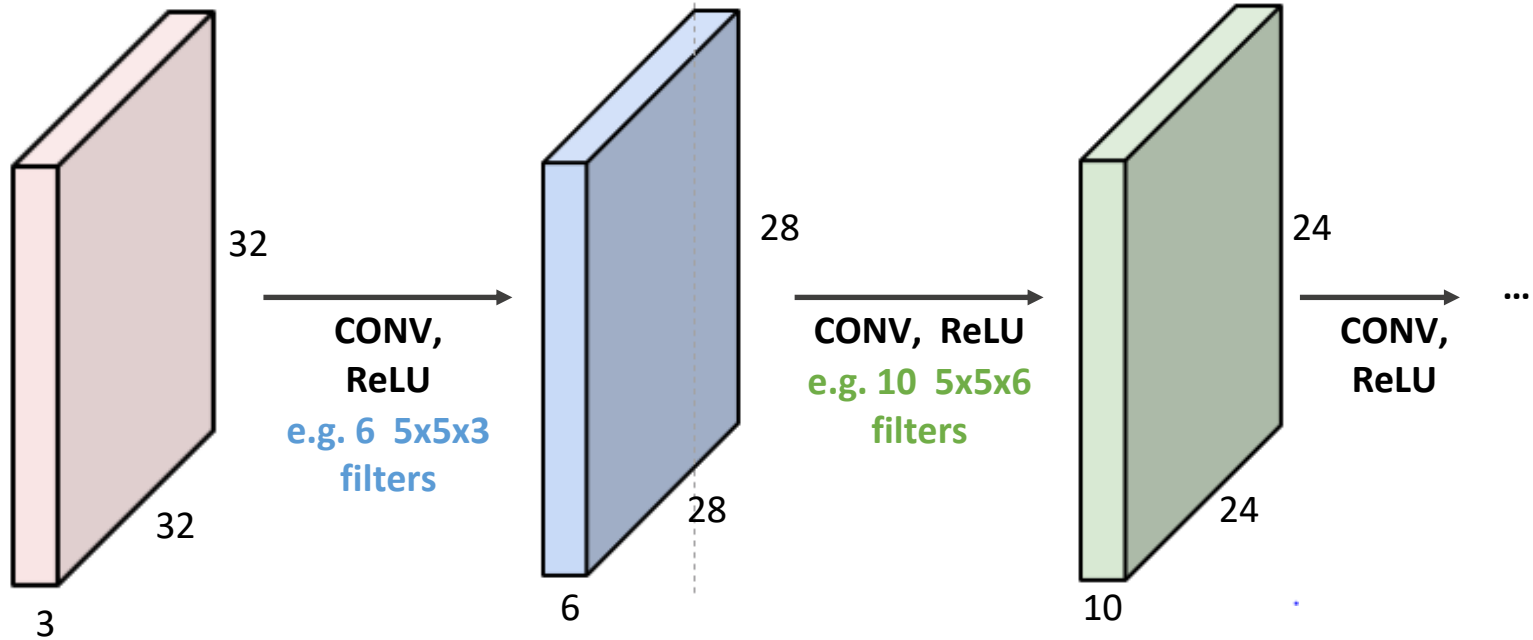


We stack these up to get a “new image” of size
28x28x6!

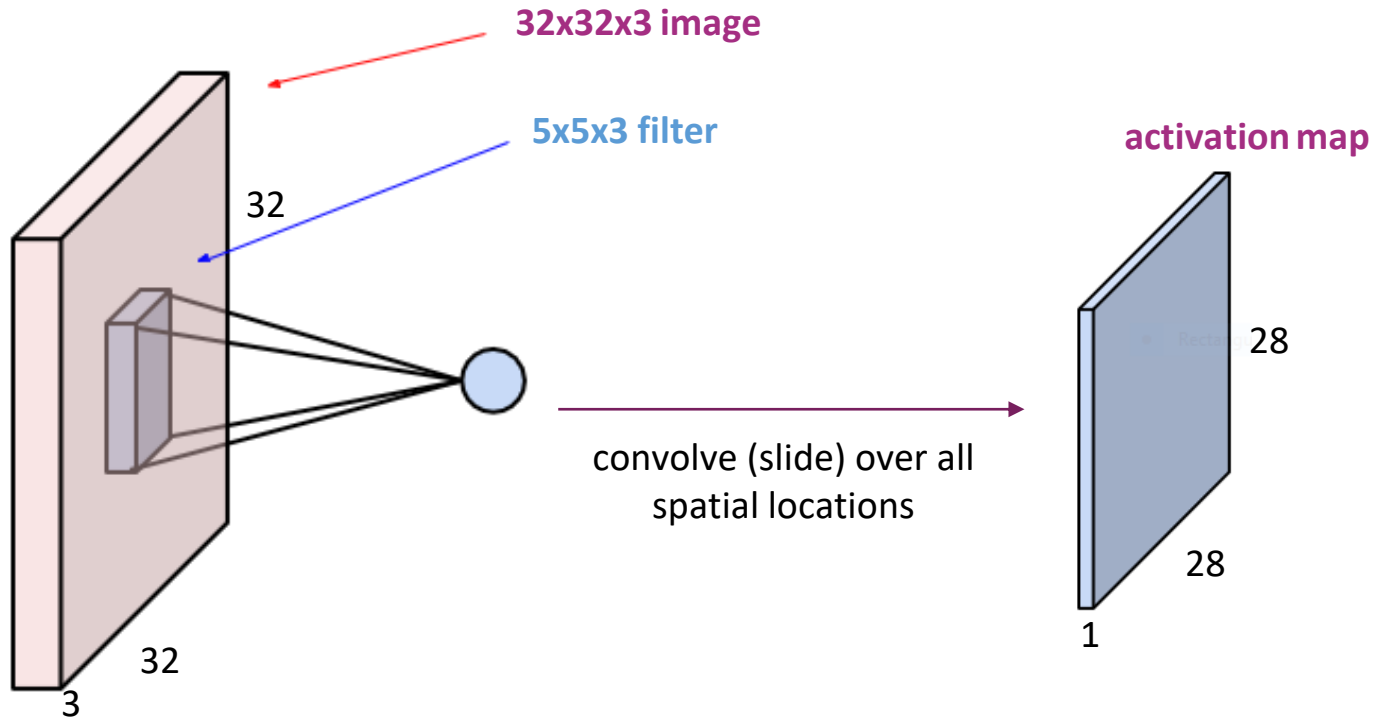
Preview: ConvNet is a sequence of Convolution Layers, interspersed with activation functions



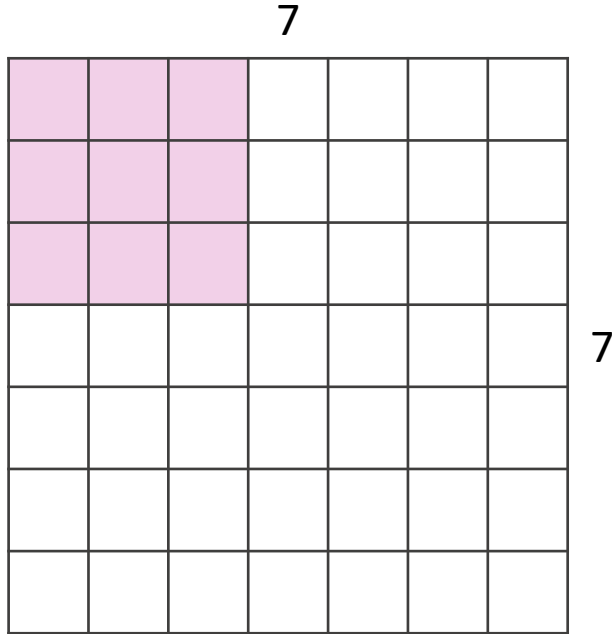
Preview: ConvNet is a sequence of Convolution Layers, interspersed with activation functions



A closer look at spatial dimensions:

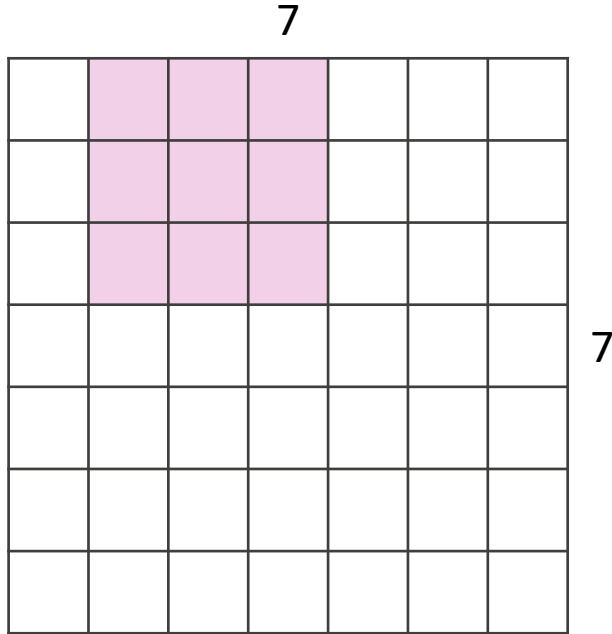


A closer look at spatial dimensions:



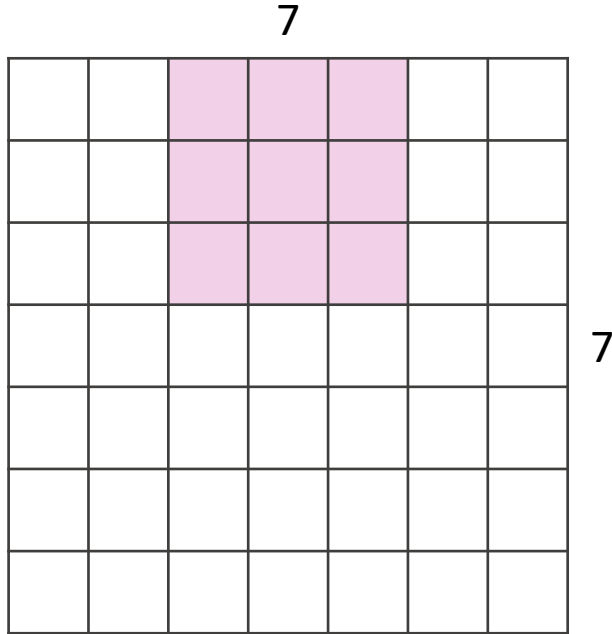
7x7 input (spatially)
assume 3x3 filter

A closer look at spatial dimensions:



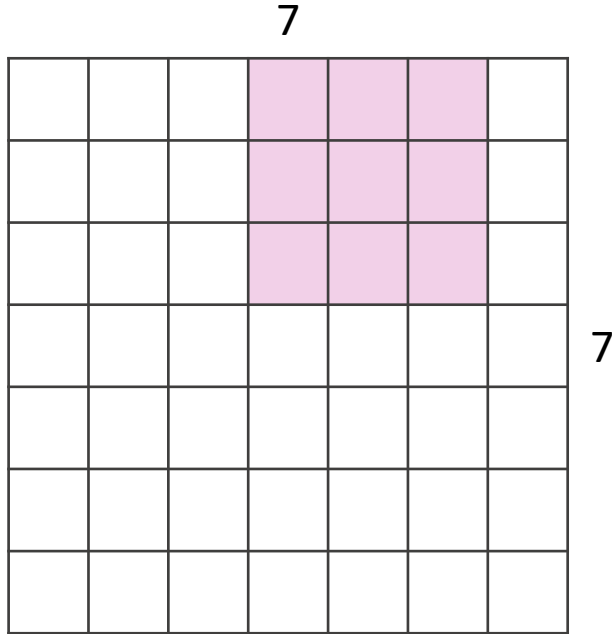
7x7 input (spatially)
assume 3x3 filter

A closer look at spatial dimensions:



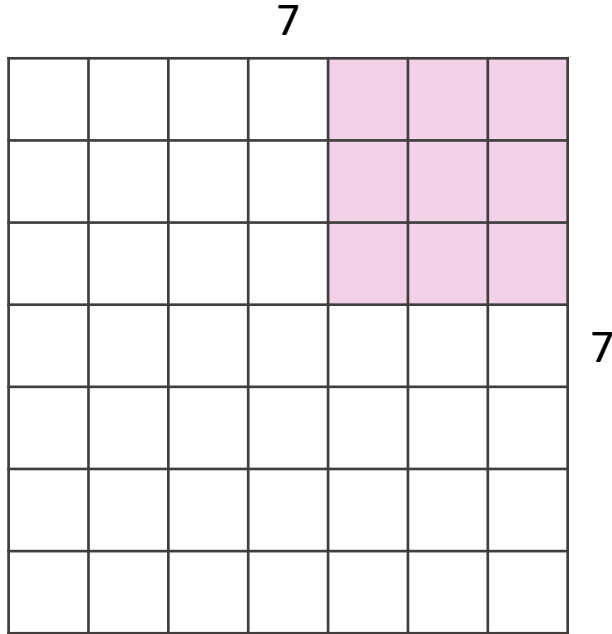
7x7 input (spatially)
assume 3x3 filter

A closer look at spatial dimensions:



7x7 input (spatially)
assume 3x3 filter

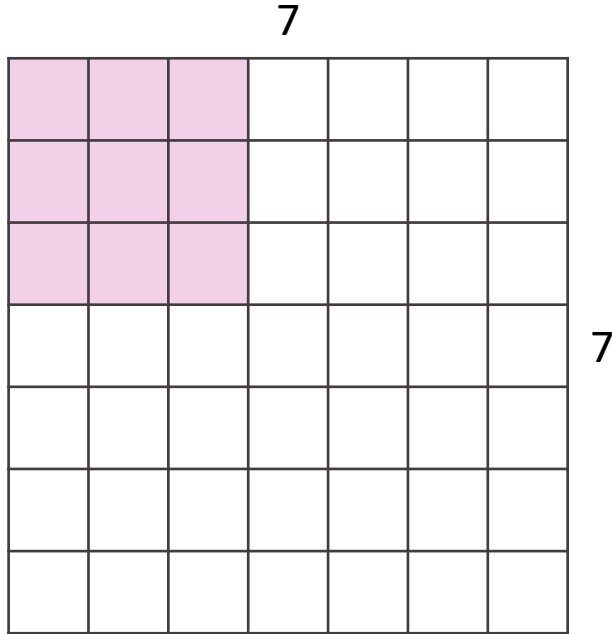
A closer look at spatial dimensions:



**7x7 input (spatially)
assume 3x3 filter**

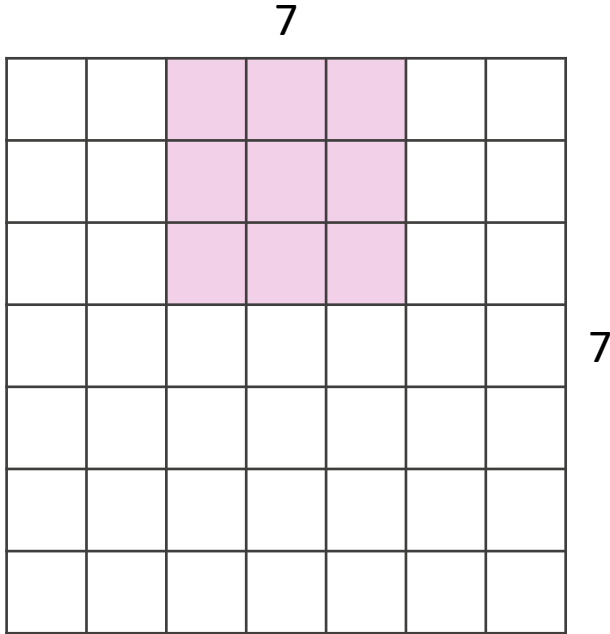
=> 5x5 output

A closer look at spatial dimensions:



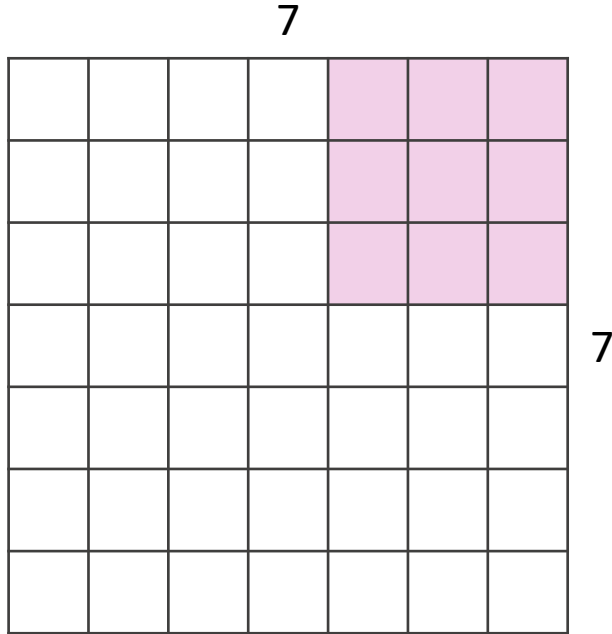
7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**

A closer look at spatial dimensions:



7x7 input (spatially)
 assume 3x3 filter
 applied **with stride 2**

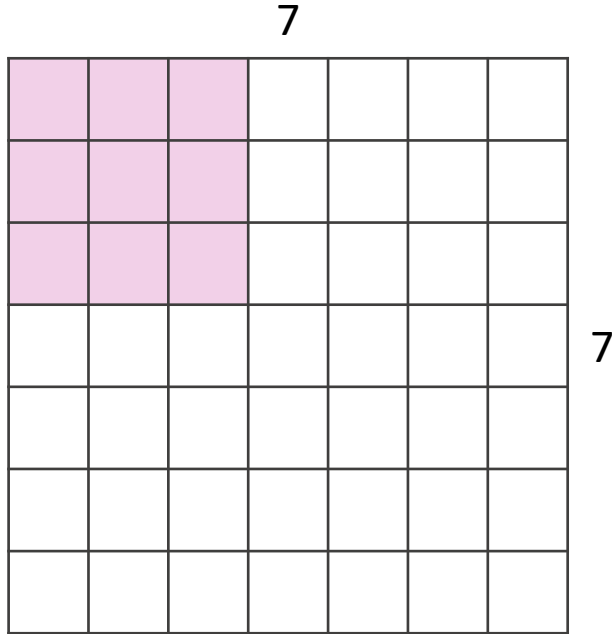
A closer look at spatial dimensions:



7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**

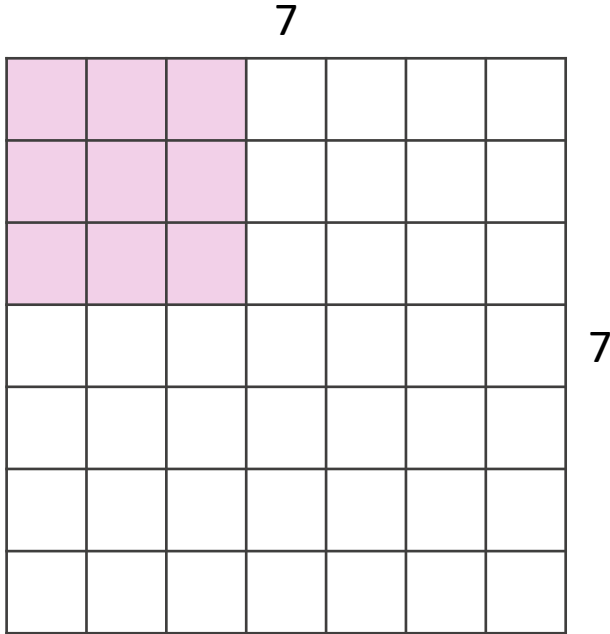
=> **3x3 output!**

A closer look at spatial dimensions:



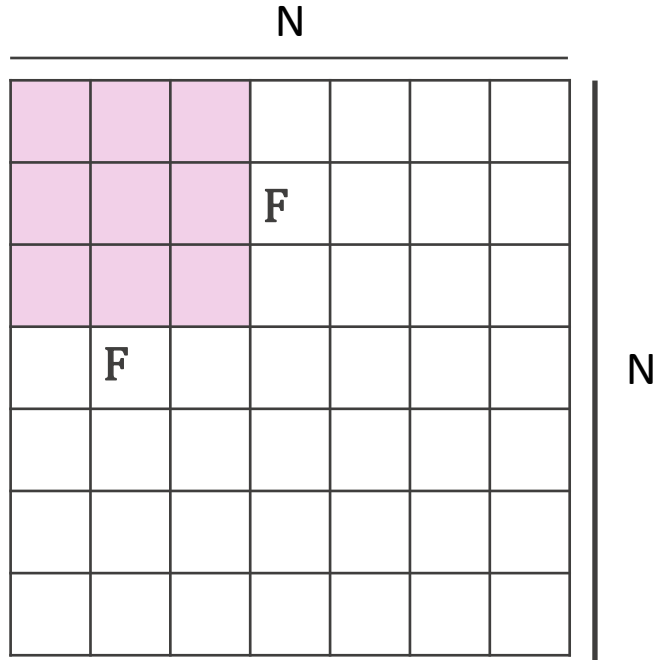
7x7 input (spatially)
 assume 3x3 filter
 applied **with stride 3?**

A closer look at spatial dimensions:



7x7 input (spatially)
assume 3x3 filter
applied **with stride 3?**

doesn't fit!
cannot apply 3x3 filter on
7x7 input with stride 3.



Output size:

$$(N - F) / \text{stride} + 1$$

e.g. $N = 7, F = 3$:

stride 1 $\Rightarrow (7 - 3) / 1 + 1 = 5$

stride 2 $\Rightarrow (7 - 3) / 2 + 1 = 3$

stride 3 $\Rightarrow (7 - 3) / 3 + 1 = 2.33!$

0	0	0	0	0	0			
0								
0								
0								
0								

e.g. input 7x7

3x3 filter, applied with **stride 1**

pad with 1 pixel border => what is the output?

7x7 output!

(recall:)

$$(N + 2P - F) / \text{stride} + 1$$

0	0	0	0	0	0			
0								
0								
0								
0								

e.g. input 7x7

3x3 filter, applied with **stride 1**

pad with 1 pixel: get 7x7 output

In general, common to see CONV layers with stride 1, filters of size $F \times F$, and zero-padding with $(F-1)/2$.
(will preserve size spatially)

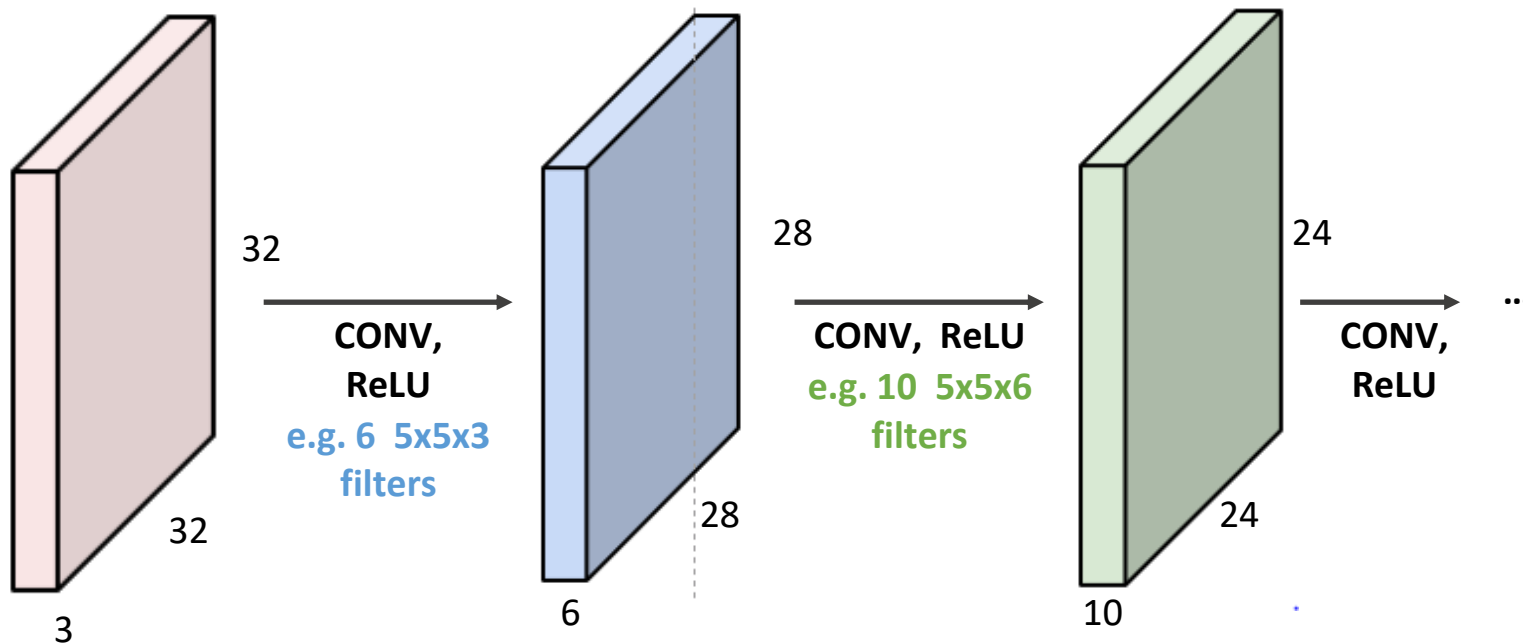
e.g. $F = 3 \Rightarrow$ zero pad with 1

$F = 5 \Rightarrow$ zero pad with 2

$F = 7 \Rightarrow$ zero pad with 3

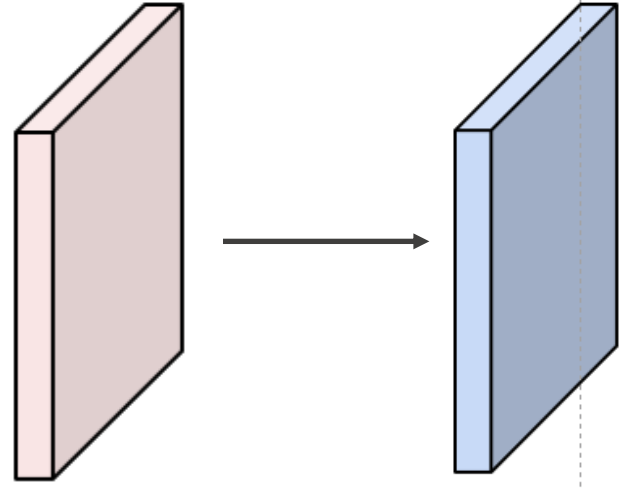
Example

E.g. 32x32 input convolved repeatedly with 5x5 filters shrinks volumes spatially! (32 -> 28 -> 24 ...). Shrinking too fast is not good and it doesn't work well.



Input volume: **32x32x3**
10 5x5 filters with stride 1, pad 2

Output volume size: ?



Example

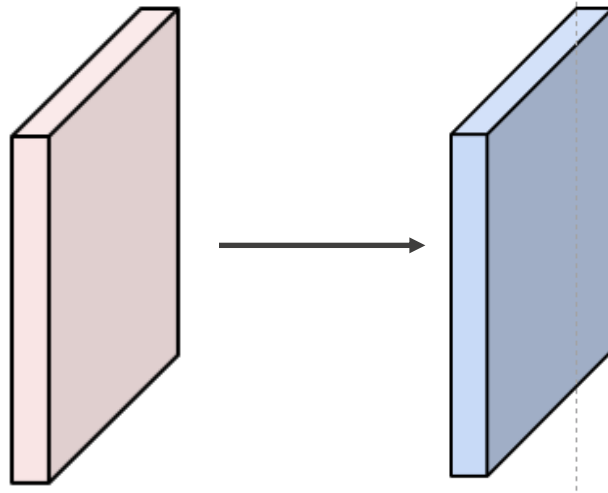
Input volume: **32x32x3**

10 **5x5** filters with stride **1**, pad **2**

Output volume size:

$(32 + 2 * 2 - 5) / 1 + 1 = 32$ spatially, so

32x32x10

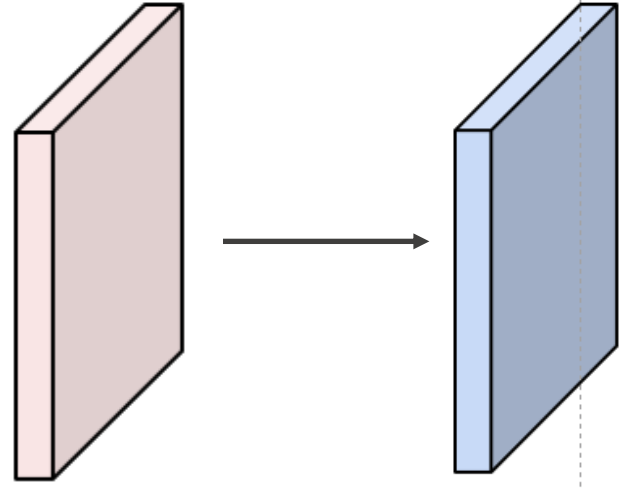


Example

Input volume: **32x32x3**

10 5x5 filters with stride 1, pad 2

Number of parameters in this layer?



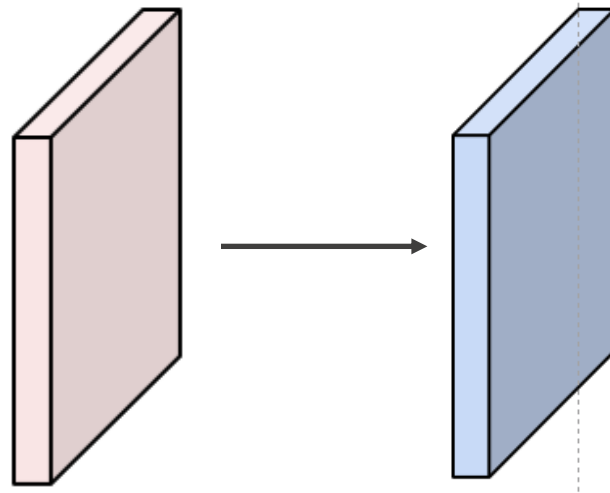
Example

Input volume: **32x32x3**

10 **5x5** filters with stride 1, pad 2

Number of parameters in this layer? each filter has $5*5*3 + 1 = 76$ params

$\Rightarrow 76*10 = 760$



(+1 for bias)

Convolution layer: summary

Let's assume input is $W_1 \times H_1 \times C$ Conv layer needs 4 hyperparameters :

- Number of filters **K**
- The filter size **F**
- The stride **S**
- The zero padding **P**

This will produce an output of $W_2 \times H_2 \times K$ where :

- $W_2 = (W_1 - F + 2P)/S + 1$
- $H_2 = (H_1 - F + 2P)/S + 1$

Number of parameters: F^2CK and K biases

Convolution layer: summary

Let's assume input is $W_1 \times H_1 \times C$ Conv layer needs 4 hyperparameters :

- Number of filters **K**
- The filter size **F**
- The stride **S**
- The zero padding **P**

This will produce an output of $W_2 \times H_2 \times K$ where :

- $W_2 = (W_1 - F + 2P)/S + 1$
- $H_2 = (H_1 - F + 2P)/S + 1$

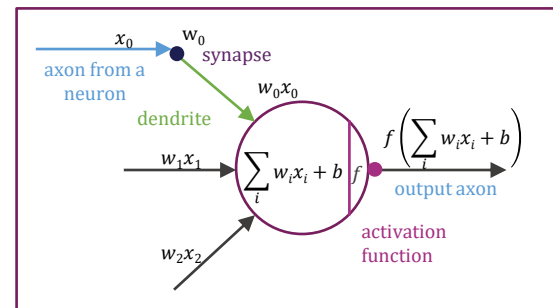
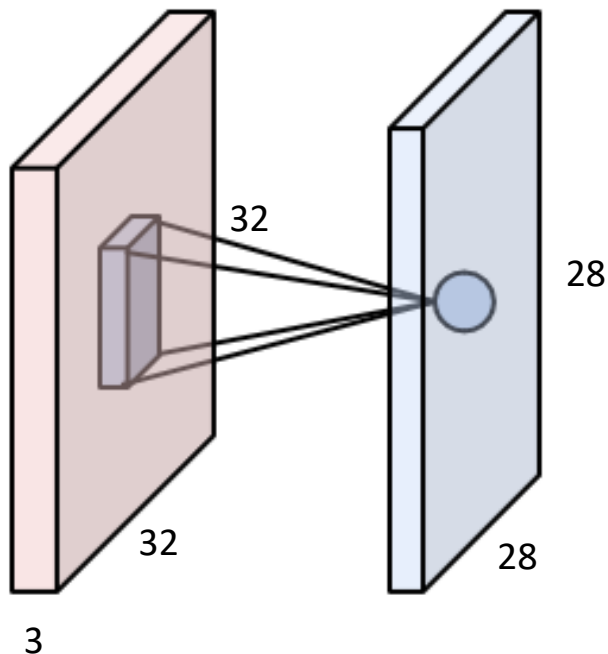
Number of parameters: F^2CK and K biases

Common settings :

K = (powers of 2, e.g. 32, 64, 128, 512)

- $F = 3, S = 1, P = 1$
- $F = 5, S = 1, P = 2$
- $F = 1, S = 1, P = 0$

The brain/neuron view of CONV Layer

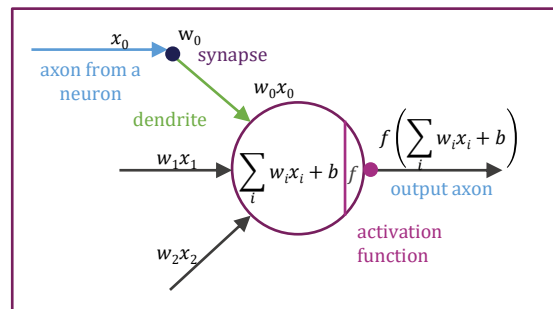
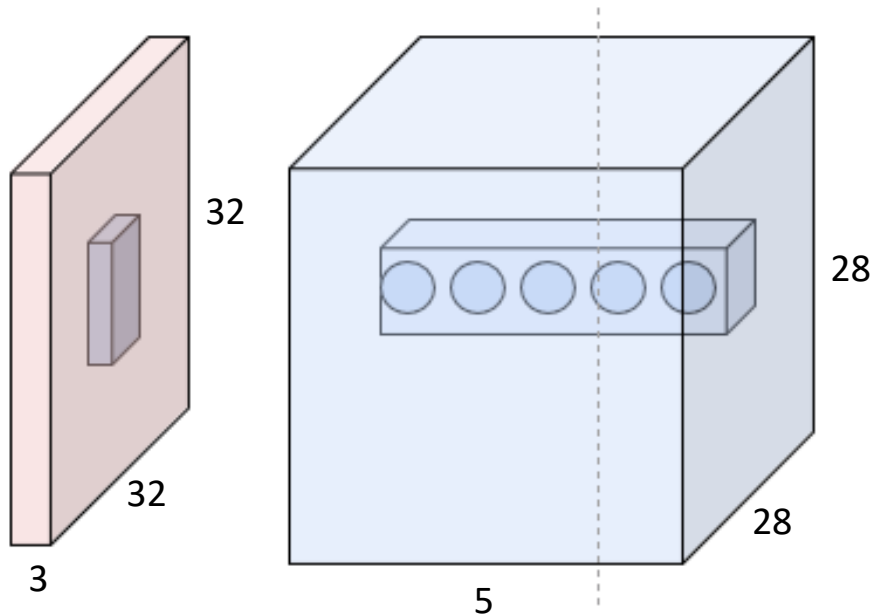


An activation map is a 28x28 sheet of neuron outputs :

- Each is connected to a small region in the input
- All of them share parameters

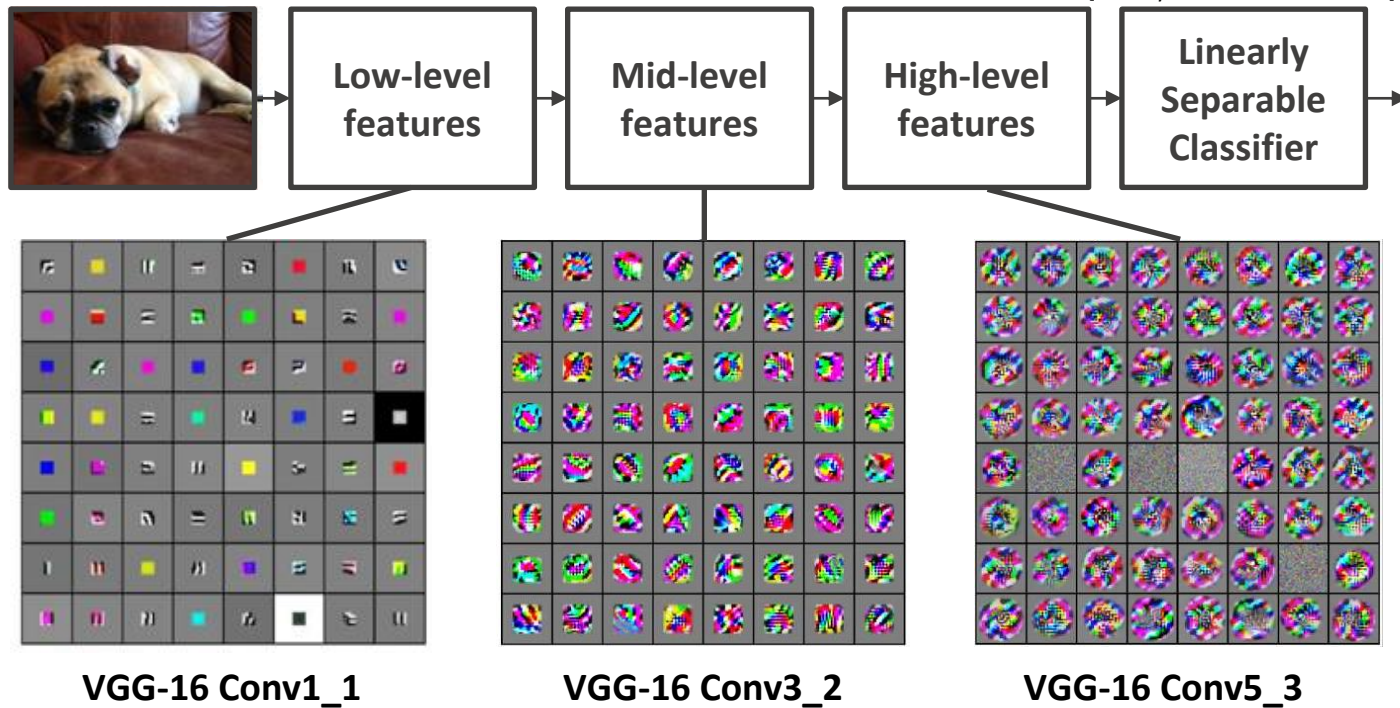
“5x5 filter” -> “5x5 receptive field for each neuron”

The brain/neuron view of CONV Layer



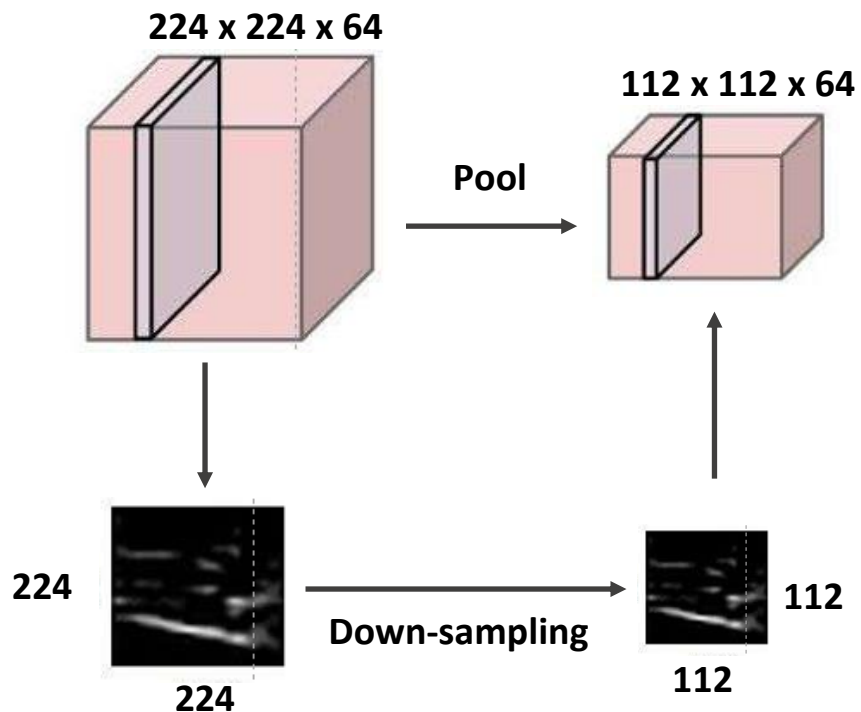
- E.g. with 5 filters, CONV layer consists of neurons arranged in a 3D grid (28x28x5)
- There will be 5 different neurons all looking at the same region in the input volume

[Zeiler and Fergus 2013] Visualization of VGG-16 by Lane McIntosh. VGG-16 architecture from [Simonyan and Zisserman 2014].



Pooling layer

- Makes the representations smaller and more manageable
- Operates over each activation map independently



MAX POOLING

Single depth slice

1	1	2	4
5	6	7	8
3	2	1	0
1	2	3	4

Max pool with 2x2
filters and stride 2



6	8
3	4

Pooling layer: Summary

Let's assume input is $W_1 \times H_1 \times C$, applying Pooling operation would need 2 hyperparameters :

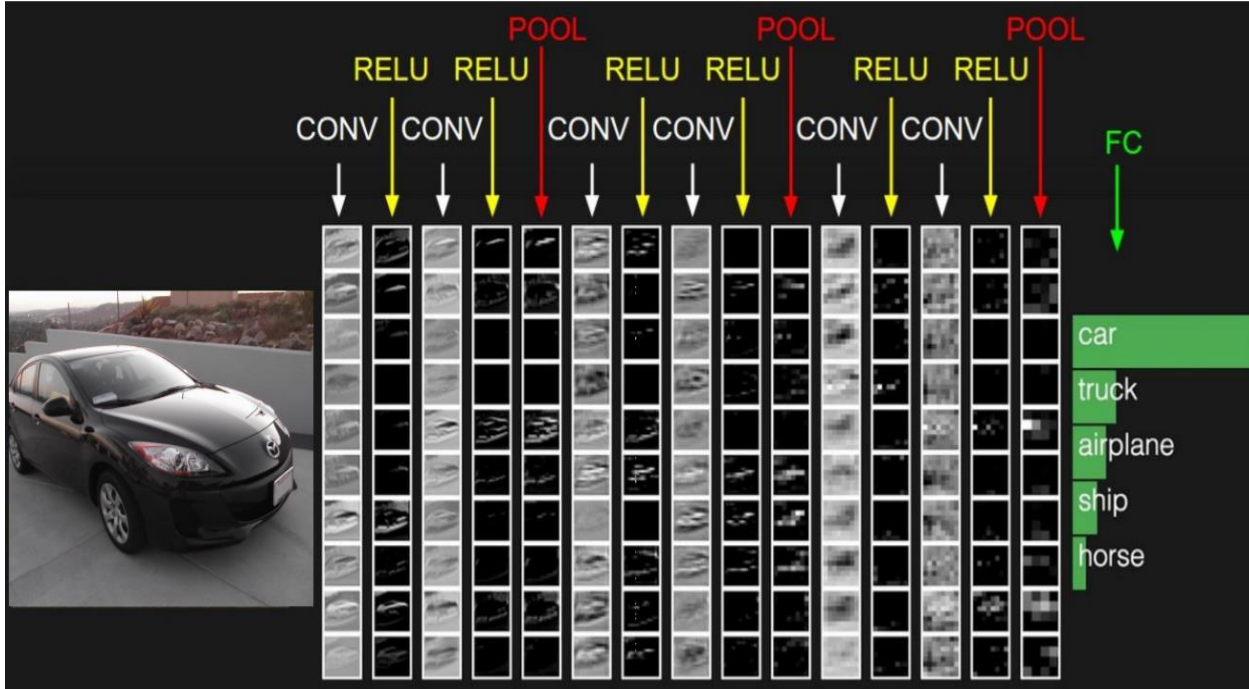
- The spatial extent F
- The stride S

This will produce an output of $W_2 \times H_2 \times C$ where :

- $W_2 = (W_1 - F) / S + 1$
- $H_2 = (H_1 - F) / S + 1$

Number of parameters : 0

Combining the building blocks: Architecture Summary



$$L_i = -\log\left(\frac{e^{sy_i}}{\sum_j e^{s_j}}\right)$$



Thank you

Courtesy:

Fei Fei Li, Andrej Karpathy, Ranjay Krishna,
Alexander Amini and Sergey Levine